



CSLLT Individual Assignment

Point of Sale system

Module Code	:	Computer System Low Level Techniques (CT073-3-2-CSLLT)
Intake Code	:	APD2F2509CS(AI)
Lecturer Name	:	DR. Hassan Jamil Syed

Student ID	Student Name
TP084440	Mazen Lasheen

Table of Contents

1. Introduction	4
2. Research and Analysis	5
2.1 Overview of Assembly Language Usage in Industry	5
2.2 Assembly Language in Cybersecurity.....	5
2.3 Malware Analysis and Reverse Engineering.....	6
3. System Design	7
3.1 System Overview.....	7
3.2 System Flowchart.....	9
4. System Screenshots	11
4.1 Category Menu and Item Selection.....	11
4.2 Per-Item Add-On Customisation Session	11
4.3 Empty Order Validation	12
4.4 Coupon Code and Receipt.....	12
4.5 Input Validation.....	13
5. Source Code with Explanation	13
5.1 Architecture and Conventions.....	14
5.2 Data Section	15
5.3 Print Macro and print_item_hdr Subroutine.....	16
5.4 Main Loop and Category Dispatch.....	17
5.5 Per-Item Add-On Session Loop.....	18
5.6 Coupon Card System	19
5.7 Empty Order Validation	19
5.8 Arithmetic Operations.....	20
5.8.1 Multiplication — Item Cost.....	20
5.8.2 Addition — Accumulating the Total	20

5.8.3 Division — Discount and Display	20
5.9 Input Handling (read_line).....	21
5.10 String Conversion Subroutines.....	22
6. User Manual	23
6.1 System Requirements.....	23
6.2 Build and Run.....	23
6.3 Operating the POS System.....	23
7. Conclusion	25
8. References.....	26
Appendix A: Full Source Code	27

1. Introduction

Low-level programming language, Assembly language. Every instruction in assembly corresponds to one nearly directly implemented by the machine instruction that is executed by the processor. This is contrasted with programming languages such as Python or Java. The languages obscure the programmer from most of the features of the hardware. The programmer can use Assembly language to access registers, memory locations and CPU operations.

The x86 architecture was first introduced by Intel in 1978 with the 8086 processor. It is one of the most popular instruction set architectures in computer history. Over time, x86 evolved from 16-bit to 32-bit (IA-32) and then to 64-bit (x86-64). The 32-bit protected mode provided new capabilities such as memory protection and improved processor control. These features have become common in today's operating systems.

Computing became more abstract – the gap between the programmer and the hardware slowly increased. Compilers, runtimes and managed memory made development quicker - and at the same time layered layers of abstraction over what the machine had been doing. All of that is preceded by Assembly language. Does not simplify. It does not hide. The code that the programmer writes is essentially the code that is executed by the processor.

Even though high-level languages are used for most software development today, assembly language is still important in certain areas. These include instances when hardware control, maximum performance, or precise timing is needed.

2. Research and Analysis

2.1 Overview of Assembly Language Usage in Industry

Today, a number of technical areas continue to use assembly language. Primarily, the need for direct control of the hardware, maximum performance, and timing that cannot be reliably provided by the compiler.

An area of commonality is embedded systems and the Internet of Things (IoT). In most cases, some microcontrollers have a very small memory. A particular device may, for instance, contain 1 KB of flash memory and 64 bytes of RAM. This is too small for a C runtime environment. Note that Microchip manufacturers have application notes available for a variety of applications that utilize assembly language for bootloaders and interrupt handlers. These routines need to be performed in a fixed number of clock cycles. A compiler might introduce a time delay which could cause the system to miss its timing requirements.

Even the most fundamental low level operations of operating system kernels rely on assembly language. There are assembly files in the Linux kernel in the `arch/x86/boot/` directory. It's the code that takes the processor from 16-bit real mode to 32-bit protected mode when the computer boots. This sequence can't be encoded in C as C needs a stack to work. The stack does not come into existence until it has been set up in the assembly prologue.

To utilize SIMD instructions, the programmer may have to write instructions in assembly language. Single Instruction Multiple Data (SIMD) is the acronym. These instructions perform several operations on several data simultaneously. Hand written x86 SIMD assembly with SSE, AVX, or AVX-512 may be 4-16 times faster than regular scalar C program. It is not uncommon for libraries, such as Intel's Math Kernel Library, to have many lines of hand-optimised assembly code for the most performance-critical loops.

2.2 Assembly Language in Cybersecurity

There is a very direct relationship between assembly language and cybersecurity. Arbitrary code execution is one of the many vulnerabilities in software. These include buffer overflows, use-after-free bugs, and format string bugs. To exploit these security holes, one has to have a pretty good

knowledge of how machine code runs. The attacker must be familiar with the structure of the call stack, and know how to manipulate the processor control flow at the instruction level.

It's impossible to write exploits or do penetration testing without assembly knowledge. The writer of the shellcode must ensure that the payload does not contain any null bytes, stack is properly aligned, and system calls are made directly. Library functions cannot be used by the attacker as these functions might not exist in the attacked process. Shellcode templates can be created using tools such as Metasploit Framework or pwntools, and they are compatible with NASM.

An exploitation technique that is widely used is Return-Oriented Programming (ROP). It will be used if Data Execution Prevention is turned on. ROP involves identifying short sequences of instructions known as gadgets. Each gadget ends in a ret instruction. The attacker is able to connect these gadgets in series to get arbitrary computation. ROPgadget produces raw x86 mnemonic. Analyst has to review the register effects and stacking of each gadget.

2.3 Malware Analysis and Reverse Engineering

Malware analysis involves study of the code to gain insight into the malicious activity of malware. Malware creators do not publish the source code, so the analysts have to use the code in machine language. This translates to working at the assembly level. Typically the field is split in two parts, static analysis and dynamic analysis. Static analysis is simply reading the disassembly (in tools such as IDA Pro and Ghidra). Dynamic analysis involves going through the code in a debugger such as x64dbg.

One of the well-known examples is the analysis of the WannaCry ransomware from 2017. The domain check for a killswitch was discovered by examining the assembly code. They detected a conditional jump after an API call. This was valuable information that enabled somebody to register that domain and thwart the attack. There is also an aspect of digital forensics that involves assembly knowledge. Tools such as Volatility are used by investigators to extract shellcode from memory dumps. YARA rules are compared to a specific sequence of bytes, a sequence that is generated by the assembly instructions.

3. System Design

3.1 System Overview

The system is a console-based Point of Sale application for a small food and beverage canteen. It allows the operator to navigate a four-category menu, select items with quantities, customise each unit with add-ons, apply a coupon code, and receive a formatted receipt. The system mainly uses three concepts to function:

- Arithmetic operations: `imul` for price-quantity multiplication, `add` for running total accumulation, `div` for discount and display formatting, and `sub` for final payable calculation.
- Loops: The main category loop repeats via `jmp main_loop`. Each per-item add-on session loops via `jmp` to its own `addon_loop` label. The `item_counter` loop repeats the session for every unit of the ordered quantity.
- Conditional jumps: `je`, `jl`, `jg` branch control flow for category dispatch, item validation, add-on validation, discount triggering, coupon matching, and empty-order detection.

The product catalogue comprises eight items across four categories:

Category	Code	Item	Base Price (RM)	Add-ons Available
Coffee	1	Espresso	4.00	Extra Shot (+RM2.00), Sugar (+RM0.50), Whipped Cream (+RM1.00)
Coffee	2	Latte	6.00	Whipped Cream (+RM1.00), Caramel Syrup (+RM1.00), Extra Shot (+RM2.00)
Food	1	Sandwich	10.00	Add Cheese (+RM1.00), Toasted (+RM0.50)
Food	2	Cake	12.00	Whipped Cream (+RM1.00), Ice Cream (+RM2.00)
Juice	1	Orange Juice	6.00	Add Ice (+RM0.50), Large Size (+RM2.00)
Juice	2	Mango Juice	8.00	Add Ice (+RM0.50), Add Sago (+RM1.00), Large Size (+RM2.00)
Water	1	Mineral	2.00	Large Size (+RM1.00), Add Lemon (+RM0.50)
Water	2	Sparkling	3.00	Add Lime (+RM0.50), Large Size (+RM1.00)

All prices are stored in sen (1 RM = 100 sen) as 32-bit dd values to avoid floating-point arithmetic. Four price arrays (coffee_prices, food_prices, juice_prices, water_prices) and eight add-on arrays are accessed by zero-based index using scaled addressing ([array + ecx*4]).

Trigger / Code	Discount Type	Amount	Stacks With
Total >= RM 100.00	Automatic	10% off subtotal	Any coupon code
SAVE10	Coupon code	10% off subtotal	Auto-discount
FLAT5	Coupon code	RM 5.00 flat	Auto-discount
VIP20	Coupon code	20% off subtotal	Auto-discount

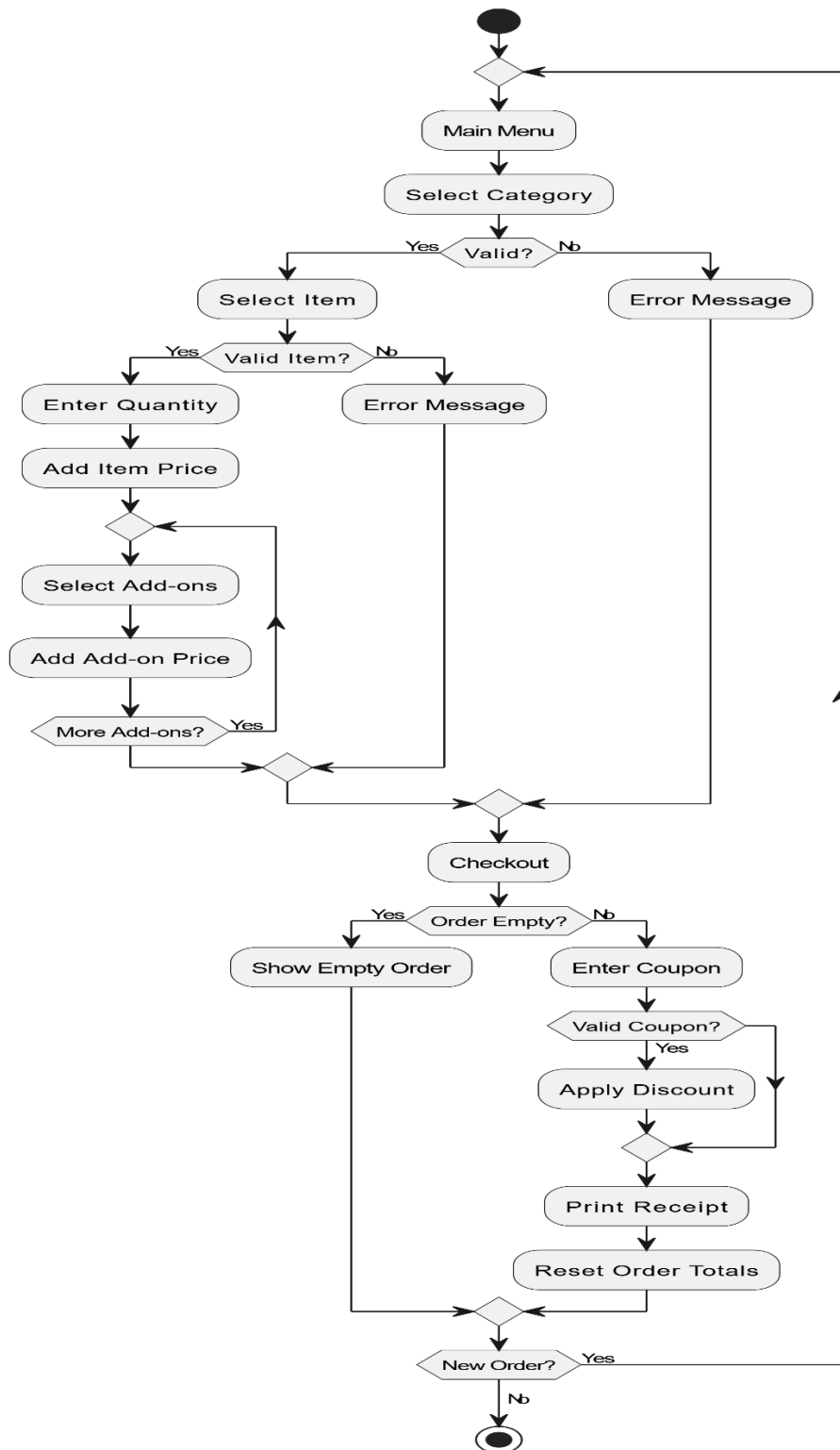
3.2 System Flowchart

Flowchart begins at the Start of the program. The welcome banner is the first to be printed. After that the program goes into the main loop. Prints the category menu and awaits input in main loop. When the user types 0 the program looks to see if something was ordered. If no items have been ordered an error message is printed and the program goes back to the main loop. If any items have been ordered, then the program proceeds to the coupon section.

The user takes a number as input; if he/she types 1, 2, 3, or 4, the program moves to the corresponding category section. The menu for each category is printed separately and then awaits an item selection. If the user types 0, the program will go back to the main loop. Once an item code is entered, the program will prompt the user for a quantity. Once validated, the price of the base is multiplied by the quantity and the value is added to the total.

Then the program goes into a per-item add-on session. The number of items is set on a counter. For each unit, the program prints an Item X of Y header and shows the add-on menu. The program will test the bitmask before applying an add-on. If the bit for the add-on is already set, an error message is displayed. Otherwise, the add-on price is added and the bit is set. The user types 0 when he/she is done with that unit. The counter is reduced and then the process is repeated with a new bitmask if there are still units left in the counter.

If the user enters 0 at the main menu, when the total is not 0, then the coupon prompt is displayed. Three valid codes are compared with the code entered. A match will be recorded if found, otherwise, the coupon discount will not be recorded. The receipt is then printed with the subtotal, auto-discount (if any), coupon discount (if any) and the final total. Once received, the program prompts to begin a new order or quit.



4. System Screenshots

4.1 Category Menu and Item Selection

The operator selects a category (1-4), then a specific item. Entering 0 at the item prompt returns to the category menu.

```
mezzo@DESKTOP-M6ABVF5:/mnt/c/Users/mezzo/OneDrive/Desktop/asm$ ./pos_system
=====
Welcome to POS
=====

=====
SELECT CATEGORY
=====
1. Coffee
2. Food
3. Juice
4. Water
5. Exit
0. Done / View Receipt
Select (0-5): 1

--- Coffee ---
1. Espresso      RM  4.00
2. Latte         RM  6.00
0. Back
Select item (0-2):
```

4.2 Per-Item Add-On Customisation Session

After entering quantity 3, the system enters three separate customisation sessions, each Espresso item (out of 3) has its own customization menu in case the customer wants to have add-ons to their order. If the same add-on is selected twice in the same session, it is rejected.

```
--- Coffee ---
1. Espresso      RM  4.00
2. Latte         RM  6.00
0. Back
Select item (0-2): 1
Enter quantity   : 3
Item added.

--- Item 1 of 3 ---

--- Espresso Add-ons ---
1. Extra Shot    +RM  2.00
2. Sugar         +RM  0.50
3. Whipped Cream +RM  1.00
0. Done
Select add-on (0-3): 1

--- Espresso Add-ons ---
1. Extra Shot    +RM  2.00
2. Sugar         +RM  0.50
3. Whipped Cream +RM  1.00
0. Done
Select add-on (0-3): 1
Add-on already selected for this item. Choose another.
```

4.3 Empty Order Validation

Pressing 0 on the main menu without ordering anything is rejected; the system returns to the category menu.

```
=====
Welcome to POS
=====

=====
SELECT CATEGORY
=====
1. Coffee
2. Food
3. Juice
4. Water
5. Exit
0. Done / View Receipt
Select (0-5): 0
No items added. Please order something first.
```

4.4 Coupon Code, discount and Receipt

After selecting 0 with a valid order, the coupon prompt appears. The receipt shows each discount line independently. The auto-discount and the coupon discount both apply when eligible.

```
Do you have a coupon? (y/n): y
Enter coupon code: VIP20

=====
RECEIPT
=====
Subtotal : RM 104.50
(10% discount applied!)
Discount : RM 10.45
(VIP20 - 20% off!)
Coupon   : RM 20.90
Total    : RM 73.15
=====

Thank you for your purchase!
=====
```

4.5 Input Validation

Invalid category or item codes print an error and the menu repeats without modifying the running total.

```
=====
                        SELECT CATEGORY
=====
1. Coffee
2. Food
3. Juice
4. Water
5. Exit
0. Done / View Receipt
Select (0-5): 9
Invalid input. Try again.

=====
                        SELECT CATEGORY
=====
1. Coffee
2. Food
3. Juice
4. Water
5. Exit
0. Done / View Receipt
Select (0-5): 1

--- Coffee ---
1. Espresso      RM  4.00
2. Latte         RM  6.00
0. Back
Select item (0-2): 8
Invalid input. Try again.
```

5. Source Code with Explanation

5.1 Architecture and Conventions

The programme targets x86 IA-32 protected mode, assembled with NASM (elf32 format) and linked with GNU ld (elf_i386 emulation) for Linux. System calls use software interrupt `int 0x80` with the syscall number in EAX and arguments in EBX, ECX, EDX in order.

Register	Purpose	Example Use in POS
EAX	Syscall number / return value / accumulator	eax=4 (sys_write), price in sen
EBX	Syscall arg 1 / item index / quantity	ebx=1 (stdout)
ECX	Syscall arg 2 / loop counter / add-on index	ecx=input_buf address
EDX	Syscall arg 3 / div remainder	edx=1 (byte count)
ESI	String pointer for _strlen	esi=msg_welcome
EDI	char_buf address in read_line	edi=char_buf

5.2 Data Section

All string constants are null-terminated byte arrays in the .data section. Category prices and add-on prices are stored in separate 32-bit integer arrays (dd). Storing values in sen (hundredths of a ringgit) as the internal unit removes the need for floating-point arithmetic throughout the programme.

```
coffee_prices    dd 400, 600
food_prices      dd 1000, 1200
juice_prices     dd 600, 800
water_prices     dd 200, 300

espresso_addons  dd 200, 50, 100
latte_addons     dd 100, 100, 200
sandwich_addons  dd 100, 50
cake_addons      dd 100, 200
orange_addons    dd 50, 200
mango_addons     dd 50, 100, 200
mineral_addons   dd 100, 50
sparkling_addons dd 50, 100
```

The .bss section reserves uninitialised storage for runtime variables.

```
section .bss
    input_buf    resb 32
    coupon_buf   resb 16
    char_buf     resb 4
    total_sen    resd 1
    discount_sen resd 1
    coupon_sen   resd 1
    payable_sen  resd 1
    num_buf      resb 16
    saved_choice resd 1
    saved_qty    resd 1
    item_counter resd 1
    coupon_type  resd 1
    addon_mask   resb 1
```

5.3 Print Macro and `print_item_hdr` Subroutine

A NASM macro abstracts the `sys_write` pattern. The macro computes the string length via `_strlen` and issues `syscall 4`.

```
%macro print_str 1
    mov     esi, %1
    call    _strlen
    mov     eax, 4
    mov     ebx, 1
    mov     ecx, %1
    int     0x80
%endmacro
```

Each of the eight items needs an "--- Item X of Y ---" header at the start of every add-on session. Rather than duplicating the nine-instruction sequence eight times (72 lines), a shared subroutine is used.

```
print_item_hdr:
    print_str msg_item_hdr1
    mov     eax, [saved_qty]
    sub     eax, [item_counter]
    inc     eax
    call    itoa_print
    print_str msg_item_hdr2
    mov     eax, [saved_qty]
    call    itoa_print
    print_str msg_item_hdr3
    ret
```

Every item's `addon_start` label begins with a single `call print_item_hdr`. The current item number is computed without division: `current = saved_qty - item_counter + 1`. This avoids duplicating the same nine instructions across eight separate locations in the code.

5.4 Main Loop and Category Dispatch

The main loop displays the category menu and dispatches to one of four section handlers using a chain of `cmp/je` instructions. This is the outermost loop construct, and it returns via `jmp main_loop` after every completed item addition:

```
; =====  
; MAIN LOOP  
; =====  
main_loop:  
    print_str msg_main_menu  
    call read_line  
    call atoi  
    cmp eax, 0  
    je show_receipt  
    cmp eax, 1  
    je coffee_section  
    cmp eax, 2  
    je food_section  
    cmp eax, 3  
    je juice_section  
    cmp eax, 4  
    je water_section  
    print_str msg_invalid  
    jmp main_loop  
  
; =====  
; COFFEE SECTION  
; =====  
coffee_section:  
    print_str msg_coffee_menu  
    call read_line  
    call atoi  
    cmp eax, 0  
    je main_loop  
    cmp eax, 1  
    jl coffee_inv  
    cmp eax, 2  
    jg coffee_inv  
    mov [saved_choice], eax  
    print_str msg_qty  
    call read_line  
    call atoi  
    cmp eax, 0  
    jle coffee_inv  
    mov [saved_qty], eax  
    mov [item_counter], eax  
    mov ecx, [saved_choice]  
    dec ecx  
    mov eax, [coffee_prices + ecx*4]  
    add [total_sen], eax  
    print_str msg_added  
    mov eax, [saved_choice]  
    cmp eax, 1  
    je espresso_addon_start  
    jmp latte_addon_start
```

5.5 Per-Item Add-On Session Loop

Each item has four labels: `addon_start` (resets the bitmask, calls `print_item_hdr`), `addon_loop` (reads and processes one add-on choice), `addon_done` (decrements `item_counter` and either returns to `main_loop` or starts the next unit), and `addon_duplicate` (rejects a repeated add-on). Espresso is shown as a representative example:

```
; --- Espresso ---
espresso_addon_start:
    mov byte [addon_mask], 0
    call print_item_hdr
espresso_addon_loop:
    print_str msg_espresso_add
    call read_line
    call atoi
    cmp eax, 0
    je espresso_done
    cmp eax, 1
    jl espresso_inv
    cmp eax, 3
    jg espresso_inv
    mov ecx, eax
    dec ecx
    movzx ebx, byte [addon_mask]
    mov edx, 1
    shl edx, cl
    test ebx, edx
    jnz espresso_duplicate
    or byte [addon_mask], dl
    mov eax, [espresso_addons + ecx*4]
    add [total_sen], eax
    jmp espresso_addon_loop
espresso_duplicate:
    print_str msg_duplicate_addon
    jmp espresso_addon_loop
espresso_done:
    mov eax, [item_counter]
    dec eax
    mov [item_counter], eax
    cmp eax, 0
    je main_loop
    mov ecx, [saved_choice]
    dec ecx
    mov eax, [coffee_prices + ecx*4]
    add [total_sen], eax
    print_str msg_added
    jmp espresso_addon_start
espresso_inv:
    print_str msg_invalid
    jmp espresso_addon_loop
```

The bitmask works by treating each bit in `addon_mask` as a flag for one add-on. When the user picks add-on N, the code shifts a 1 left by (N-1) positions and checks if that bit is already set in `addon_mask`. If it is, the selection is rejected. If not, the bit is set using `or` and the add-on price is added. The mask is reset to 0 at `addon_start`, so each unit of a multi-quantity order gets a fresh set of available add-ons. All eight items follow this same pattern.

5.6 Coupon Card System

Three coupon codes are supported: SAVE10 (10% off), FLAT5 (RM 5.00 flat), and VIP20 (20% off). Code matching uses `repe cmpsb`, a string-compare repeat prefix that decrements ECX and advances ESI/EDI while bytes are equal. After the instruction, `jne` tests for a mismatch. A boundary check (`cmp byte [coupon_buf + N], 0`) makes sure the input is not just a prefix of a longer string.

```
ask_coupon_code:
    print_str msg_coupon_ent
    call read_line
    mov esi, input_buf
    mov edi, coupon_buf
    mov ecx, 16
    rep movsb
    ; try SAVE10
    mov esi, code_save10
    mov edi, coupon_buf
    mov ecx, 6
    repe cmpsb
    jne try_flat5
    cmp byte [coupon_buf + 6], 0
    jne try_flat5
    mov dword [coupon_type], 1
    jmp do_receipt
```

The auto-discount (10% if total $\geq$ RM 100) and the coupon discount are applied as separate subtractions on the receipt. Both can apply in the same transaction.

5.7 Empty Order Validation

Before the coupon prompt, a single conditional jump guards the receipt path. If `total_sen` is zero, an error message is shown and control returns to `main_loop` without printing a receipt:

```
show_receipt:
    mov eax, [total_sen]
    cmp eax, 0
    jne coupon_check
    print_str msg_empty_order
    jmp main_loop
```

5.8 Arithmetic Operations

5.8.1 Multiplication — Item Cost

```
; Calculate price * quantity using imul
mov  ecx, [saved_choice]
dec  ecx
mov  eax, [coffee_prices + ecx*4]  ; price of one unit
mov  edx, [saved_qty]              ; quantity
imul eax, edx                      ; eax = price * quantity
```

The imul instruction multiplies the quantity by the item price in sen. The two-operand form is used so the result stays in EAX, which is then added directly to total_sen.

5.8.2 Addition — Accumulating the Total

```
add [total_sen], eax              ; add full amount to total
```

The add instruction updates total_sen directly in memory. It is used throughout the system to accumulate both base item prices and add-on prices into the running order total.

5.8.3 Division — Discount and Display

```
mov  ecx, 10
div  ecx
mov  [discount_sen], eax
print_str msg_disc_note
print_str msg_discount
mov  eax, [discount_sen]
```

The div instruction is used for discount calculations and currency formatting. EDX must be cleared before each division. The quotient goes into EAX and the remainder into EDX, which allows the ringgit and sen parts to be printed separately without using floating-point.

5.9 Input Handling (read_line)

read_line reads one byte at a time from stdin (file descriptor 0) rather than using a block read. This avoids consuming characters from the next input line when the buffer is larger than the actual input.

```
read_line:
    xor    ecx, ecx
.loop:
    mov    eax, 3
    mov    ebx, 0
    mov    edx, 1
    lea    edi, [char_buf]
    push   ecx
    mov    ecx, edi
    int    0x80
    pop    ecx
    cmp    eax, 0
    jle    .done
    movzx  eax, byte [char_buf]
    cmp    al, 10
    je     .done
    cmp    al, 13
    je     .loop
    cmp    al, 0
    je     .done
    mov    [input_buf + ecx], al
    inc    ecx
    cmp    ecx, 30
    jl     .loop
.done:
    mov    byte [input_buf + ecx], 0
    ret
```

The lea instruction loads the address of char_buf rather than its value. The movzx instruction zero-extends the 8-bit character into a 32-bit register so it can be compared against the newline (10) and carriage return (13) values correctly.

5.10 String Conversion Subroutines

5.10.1 atoi — ASCII String to Integer

The `atoi` subroutine reads the digit string in `input_buf` and converts it to an integer. It processes one digit at a time and then multiplies the accumulator by 10 and adding the new digit. For input "13", first `EAX` becomes 1, then $EAX = 1 * 10 + 3 = 13$.

```
atoi:
    xor    eax, eax
    xor    ecx, ecx
.lp:
    movzx  edx, byte [input_buf + ecx]
    cmp    dl, 0
    je     .done
    cmp    dl, '0'
    jl     .done
    cmp    dl, '9'
    jg     .done
    sub    dl, '0'
    imul   eax, eax, 10
    add    eax, edx
    inc    ecx
    jmp    .lp
.done:
    ret
```

5.10.2 itoa_print — Integer to ASCII, Print

`itoa_print` converts the value in `EAX` to a decimal string and prints it. The conversion builds the string from the right to the left in `num_buf` by repeatedly dividing by 10 and storing the remainder as an ASCII digit.

```
itoa_print:
    lea    esi, [num_buf + 12]
    mov    byte [esi], 0
    mov    ecx, 10
.lp:
    xor    edx, edx
    div    ecx
    add    dl, '0'
    dec    esi
    mov    [esi], dl
    test   eax, eax
    jnz    .lp
    lea    edx, [num_buf + 12]
    sub    edx, esi
    mov    eax, 4
    mov    ebx, 1
    mov    ecx, esi
    int    0x80
    ret
```

The test eax, eax instruction checks whether the quotient has reached zero, which means all digits have been processed. itoa_print is reused by print_item_hdr for the item number display and by print_rm for the ringgit part of the receipt.

6. User Manual

6.1 System Requirements

- Operating System: Linux (32-bit or 64-bit with 32-bit compatibility libraries)
- Assembler: NASM 2.14 or later
- Linker: GNU ld (binutils)
- WSL users: install via `sudo apt install nasm binutils`

6.2 Build and Run

To assemble and run the program, run the following command in WSL or Linux terminal:

```
nasm -f elf32 pos_system.asm -o pos_system.o
ld -m elf_i386 pos_system.o -o pos_system
./pos_system
```

6.3 Operating the POS System

Step 1: Welcome banner and category menu appear on launch.

Step 2: Enter a category (1-4) and press Enter. Enter 0 to finalise.

Step 3: Enter an item code within the category. Enter 0 to go back.

Step 4: Enter quantity (positive integer). Base price is added to the running total.

Step 5: Per-item add-on sessions begin. For each unit, select add-ons one at a time. Each add-on can only be selected once per unit. Enter 0 (Done) when finished. Sessions repeat for each unit of the quantity.

Step 6: Return to the category menu to add more items, or enter 0 to checkout.

Step 7: Enter y to apply a coupon code (SAVE10, FLAT5, or VIP20), or n to skip.

Step 8: The receipt displays the subtotal, auto-discount (if $\geq$ RM 100), coupon discount (if applied), and final total.

List of scenarios for optimal program testing:

Scenario	Expected Outcome
Enter category 0 with no items ordered	Error message; returns to category menu
Enter category 1-4	Category sub-menu appears
Enter item 0 in sub-menu	Returns to main category menu
Enter invalid category or item code	Invalid message; menu repeats
Enter quantity 0 or non-numeric	Invalid message; menu repeats
Select same add-on twice in one session	Duplicate message; prompt repeats
Total $\geq$ RM 100.00	10% auto-discount shown on receipt
Coupon SAVE10	10% coupon discount deducted
Coupon FLAT5	RM 5.00 flat deduction
Coupon VIP20	20% coupon discount deducted
Auto-discount + any coupon	Both applied independently on receipt

7. Conclusion

This was written in assembly language, which meant that all of this had to be done by hand: no standard library, no built-in method to handle input, no automatic memory management. To print a price or read a key press involves configuring registers, calling a system call, and handling the result manually. It slowed down the development process, but also facilitated understanding of exactly what the processor was doing at each step.

The system covers the basics of assembly programming arithmetic operations, loops, and conditional branching, and extends that with more features. Prices are saved in sen, not ringgit, so as to prevent floating point arithmetic as integer division will be suitable for calculating discounts as well as performing ringgit-cent division to display the price. The `read_line` subroutine reads one byte at a time, so that any characters left over in the buffer won't affect the next read. The duplicate add-on check is performed using only a single byte as a bitmask so no loop is required, one shift and one test instruction is sufficient to test or set any add-on.

The coupon system compares strings without calling any string functions (with a `repe cmpsb`) and the `print_item_hdr` subroutine handles the "Item X of Y" header that appears at the start of each add-on session. These decisions helped to keep the code cleaner and easier to read.

But there are some restrictions. Not all items are listed on the receipt, only the subtotal and any discounts are noted. Coupon codes are case-sensitive; that is, "save10" will not match with "SAVE10". There is also no file output, which means that nothing is saved between sessions.

The research portion involved topics of current interest today in assembly language like embedded systems, operating systems' start-up routines, and cyber security topics such as reverse engineering and creating exploits. Ultimately, the experience of working on this project gave a much better understanding of the rationale of using assembly.

8. References

- Alvarez, V. (2023). YARA: The pattern matching swiss knife for malware researchers. VirusTotal. <https://virustotal.github.io/yara/>
- Hyde, R. (2010). The art of assembly language (2nd ed.). No Starch Press.
- Hutchins, M. (2017). How I accidentally stopped a global cyber attack. MalwareTech Blog. <https://www.malwaretech.com/2017/05/how-i-accidentally-stopped-a-global-cyber-attacks.html>
- Intel Corporation. (2024). Intel oneAPI Math Kernel Library developer reference. Intel. <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/>
- Microchip Technology. (2023). AVR instruction set manual. Microchip Technology Inc. <https://www.microchip.com/content/dam/mchp/documents/OTH/ApplicationNotes/ApplicationNotes/AVR-InstructionSet-Manual-DS40002198.pdf>
- NASM Development Team. (2023). NASM - The Netwide Assembler (version 2.16). <https://www.nasm.us/doc/nasmdoc0.html>
- Rapid7. (2024). Metasploit framework documentation. Rapid7. <https://docs.metasploit.com/>
- Sikorski, M., & Honig, A. (2012). Practical malware analysis: The hands-on guide to dissecting malicious software. No Starch Press.
- Torvalds, L. (2024). Linux kernel source: arch/x86/boot/. GitHub. <https://github.com/torvalds/linux/tree/master/arch/x86/boot>
- Zero Day Initiative. (2023). ZDI vulnerability disclosure policy and research methodology. Trend Micro. <https://www.zerodayinitiative.com/advisories/published/>

Appendix A: Full Source Code

```
; =====
; Point of Sale (POS) System
; Architecture: x86 (32-bit), Assembler: NASM
; OS: Linux (int 0x80 syscalls)
; =====

section .data
    msg_welcome      db "=====", 10
                    db "  Welcome to POS      ", 10
                    db "=====", 10, 0

    msg_main_menu     db 10, "=====", 10
                    db "          SELECT CATEGORY          ", 10
                    db "=====", 10
                    db "1. Coffee", 10
                    db "2. Food", 10
                    db "3. Juice", 10
                    db "4. Water", 10
                    db "5. Exit", 10
                    db "0. Done / View Receipt", 10
                    db "Select (0-5): ", 0

    msg_coffee_menu    db 10, "--- Coffee ---", 10
                    db "1. Espresso      RM  4.00", 10
                    db "2. Latte          RM  6.00", 10
                    db "0. Back", 10
                    db "Select item (0-2): ", 0

    msg_espresso_add    db 10, "--- Espresso Add-ons ---", 10
                    db "1. Extra Shot    +RM 2.00", 10
                    db "2. Sugar         +RM 0.50", 10
                    db "3. Whipped Cream +RM 1.00", 10
                    db "0. Done", 10
                    db "Select add-on (0-3): ", 0

    msg_latte_add       db 10, "--- Latte Add-ons ---", 10
                    db "1. Whipped Cream +RM 1.00", 10
                    db "2. Caramel Syrup +RM 1.00", 10
                    db "3. Extra Shot    +RM 2.00", 10
                    db "0. Done", 10
                    db "Select add-on (0-3): ", 0

    msg_food_menu       db 10, "--- Food ---", 10
                    db "1. Sandwich      RM 10.00", 10
```

```
db "2. Cake          RM 12.00", 10
db "0. Back", 10
db "Select item (0-2): ", 0

msg_sandwich_add db 10, "--- Sandwich Add-ons ---", 10
db "1. Add Cheese    +RM 1.00", 10
db "2. Toasted       +RM 0.50", 10
db "0. Done", 10
db "Select add-on (0-2): ", 0

msg_cake_add     db 10, "--- Cake Add-ons ---", 10
db "1. Whipped Cream +RM 1.00", 10
db "2. Ice Cream     +RM 2.00", 10
db "0. Done", 10
db "Select add-on (0-2): ", 0

msg_juice_menu   db 10, "--- Juice ---", 10
db "1. Orange Juice  RM 6.00", 10
db "2. Mango Juice   RM 8.00", 10
db "0. Back", 10
db "Select item (0-2): ", 0

msg_orange_add   db 10, "--- Orange Juice Add-ons ---", 10
db "1. Add Ice       +RM 0.50", 10
db "2. Large Size    +RM 2.00", 10
db "0. Done", 10
db "Select add-on (0-2): ", 0

msg_mango_add    db 10, "--- Mango Juice Add-ons ---", 10
db "1. Add Ice       +RM 0.50", 10
db "2. Add Sago      +RM 1.00", 10
db "3. Large Size    +RM 2.00", 10
db "0. Done", 10
db "Select add-on (0-3): ", 0

msg_water_menu   db 10, "--- Water ---", 10
db "1. Mineral       RM 2.00", 10
db "2. Sparkling     RM 3.00", 10
db "0. Back", 10
db "Select item (0-2): ", 0

msg_mineral_add  db 10, "--- Mineral Water Add-ons ---", 10
db "1. Large Size    +RM 1.00", 10
db "2. Add Lemon     +RM 0.50", 10
db "0. Done", 10
db "Select add-on (0-2): ", 0
```

```
msg_sparkling_add db 10, "--- Sparkling Water Add-ons ---", 10
                  db "1. Add Lime      +RM 0.50", 10
                  db "2. Large Size   +RM 1.00", 10
                  db "0. Done", 10
                  db "Select add-on (0-2): ", 0

msg_qty           db "Enter quantity  : ", 0
msg_item_hdr1     db 10, "--- Item ", 0
msg_item_hdr2     db " of ", 0
msg_item_hdr3     db " ---", 10, 0
msg_added         db "Item added.", 10, 0
msg_invalid       db "Invalid input. Try again.", 10, 0
msg_duplicate_addon db "Add-on already selected for this item.
Choose another.", 10, 0
msg_empty_order   db "No items added. Please order something
first.", 10, 0

msg_receipt       db 10, "=====", 10
                  db "          RECEIPT          ", 10
                  db "=====", 10, 0
msg_subtotal      db "Subtotal : RM ", 0
msg_discount      db "Discount : RM ", 0
msg_coupon_hdr    db "Coupon   : RM ", 0
msg_total         db "Total    : RM ", 0
msg_newline       db 10, 0
msg_disc_note     db "(10% discount applied!)", 10, 0
msg_thankyou      db "=====", 10
                  db "  Thank you for your purchase! ", 10
                  db "=====", 10, 0
msg_dot           db " .", 0

msg_coupon_q      db 10, "Do you have a coupon? (y/n): ", 0
msg_coupon_ent    db "Enter coupon code: ", 0
msg_coupon_inv    db "Invalid coupon. No discount applied.", 10, 0
msg_save10_note   db "(SAVE10 - 10% off!)", 10, 0
msg_flat5_note    db "(FLAT5 - RM5.00 off!)", 10, 0
msg_vip20_note    db "(VIP20 - 20% off!)", 10, 0
code_save10       db "SAVE10", 0
code_flat5        db "FLAT5", 0
code_vip20        db "VIP20", 0
msg_new_order     db 10, "=====", 10, 0
msg_continue      db "Start new order? (y/n): ", 0

coffee_prices    dd 400, 600
food_prices       dd 1000, 1200
```

```
juice_prices      dd 600, 800
water_prices      dd 200, 300

espresso_addons   dd 200, 50, 100
latte_addons      dd 100, 100, 200
sandwich_addons   dd 100, 50
cake_addons       dd 100, 200
orange_addons     dd 50, 200
mango_addons      dd 50, 100, 200
mineral_addons    dd 100, 50
sparkling_addons  dd 50, 100
```

```
section .bss
input_buf         resb 32
coupon_buf        resb 16
char_buf          resb 4
total_sen         resd 1
discount_sen      resd 1
coupon_sen        resd 1
payable_sen       resd 1
num_buf           resb 16
saved_choice      resd 1
saved_qty         resd 1
item_counter      resd 1
coupon_type       resd 1
addon_mask        resb 1
```

```
section .text
global _start
```

```
%macro print_str 1
    mov esi, %1
    call _strlen
    mov eax, 4
    mov ebx, 1
    mov ecx, %1
    int 0x80
%endmacro
```

```
; =====
; print_item_hdr: prints "--- Item X of Y ---"
; =====
print_item_hdr:
    print_str msg_item_hdr1
    mov eax, [saved_qty]
    sub eax, [item_counter]
```

```
    inc  eax
    call itoa_print
    print_str msg_item_hdr2
    mov  eax, [saved_qty]
    call itoa_print
    print_str msg_item_hdr3
    ret

; =====
_start:
    mov  dword [total_sen], 0
    mov  dword [coupon_type], 0
    mov  dword [discount_sen], 0
    mov  dword [coupon_sen], 0
    print_str msg_welcome

; =====
; MAIN LOOP
; =====
main_loop:
    print_str msg_main_menu
    call read_line
    call atoi
    cmp  eax, 0
    je   show_receipt
    cmp  eax, 1
    je   coffee_section
    cmp  eax, 2
    je   food_section
    cmp  eax, 3
    je   juice_section
    cmp  eax, 4
    je   water_section
    cmp  eax, 5
    je   exit_program
    print_str msg_invalid
    jmp  main_loop

; =====
; COFFEE SECTION
; =====
coffee_section:
    print_str msg_coffee_menu
    call read_line
    call atoi
    cmp  eax, 0
```

```
je    main_loop
cmp   eax, 1
jl    coffee_inv
cmp   eax, 2
jg    coffee_inv
mov   [saved_choice], eax
print_str msg_qty
call  read_line
call  atoi
cmp   eax, 0
jle   coffee_inv
mov   [saved_qty], eax
mov   [item_counter], eax
mov   ecx, [saved_choice]
dec   ecx
mov   eax, [coffee_prices + ecx*4]
add   [total_sen], eax
print_str msg_added
mov   eax, [saved_choice]
cmp   eax, 1
je    espresso_addon_start
jmp   latte_addon_start
coffee_inv:
print_str msg_invalid
jmp   coffee_section

; --- Espresso ---
espresso_addon_start:
mov   byte [addon_mask], 0
call  print_item_hdr
espresso_addon_loop:
print_str msg_espresso_add
call  read_line
call  atoi
cmp   eax, 0
je    espresso_done
cmp   eax, 1
jl    espresso_inv
cmp   eax, 3
jg    espresso_inv
mov   ecx, eax
dec   ecx
movzx ebx, byte [addon_mask]
mov   edx, 1
shl   edx, cl
test  ebx, edx
```



```
    jnz espresso_duplicate
    or  byte [addon_mask], dl
    mov  eax, [espresso_addons + ecx*4]
    add  [total_sen], eax
    jmp  espresso_addon_loop
espresso_duplicate:
    print_str msg_duplicate_addon
    jmp  espresso_addon_loop
espresso_done:
    mov  eax, [item_counter]
    dec  eax
    mov  [item_counter], eax
    cmp  eax, 0
    je   main_loop
    mov  ecx, [saved_choice]
    dec  ecx
    mov  eax, [coffee_prices + ecx*4]
    add  [total_sen], eax
    print_str msg_added
    jmp  espresso_addon_start
espresso_inv:
    print_str msg_invalid
    jmp  espresso_addon_loop

; --- Latte ---
latte_addon_start:
    mov  byte [addon_mask], 0
    call print_item_hdr
latte_addon_loop:
    print_str msg_latte_add
    call read_line
    call atoi
    cmp  eax, 0
    je   latte_done
    cmp  eax, 1
    jl   latte_inv
    cmp  eax, 3
    jg   latte_inv
    mov  ecx, eax
    dec  ecx
    movzx ebx, byte [addon_mask]
    mov  edx, 1
    shl  edx, cl
    test ebx, edx
    jnz  latte_duplicate
    or   byte [addon_mask], dl
```

```
        mov  eax, [latte_addons + ecx*4]
        add  [total_sen], eax
        jmp  latte_addon_loop
latte_duplicate:
        print_str msg_duplicate_addon
        jmp  latte_addon_loop
latte_done:
        mov  eax, [item_counter]
        dec  eax
        mov  [item_counter], eax
        cmp  eax, 0
        je   main_loop
        mov  ecx, [saved_choice]
        dec  ecx
        mov  eax, [coffee_prices + ecx*4]
        add  [total_sen], eax
        print_str msg_added
        jmp  latte_addon_start
latte_inv:
        print_str msg_invalid
        jmp  latte_addon_loop

; =====
; FOOD SECTION
; =====
food_section:
        print_str msg_food_menu
        call read_line
        call atoi
        cmp  eax, 0
        je   main_loop
        cmp  eax, 1
        jl   food_inv
        cmp  eax, 2
        jg   food_inv
        mov  [saved_choice], eax
        print_str msg_qty
        call read_line
        call atoi
        cmp  eax, 0
        jle  food_inv
        mov  [saved_qty], eax
        mov  [item_counter], eax
        mov  ecx, [saved_choice]
        dec  ecx
        mov  eax, [food_prices + ecx*4]
```

```
    add [total_sen], eax
    print_str msg_added
    mov eax, [saved_choice]
    cmp eax, 1
    je sandwich_addon_start
    jmp cake_addon_start
food_inv:
    print_str msg_invalid
    jmp food_section

; --- Sandwich ---
sandwich_addon_start:
    mov byte [addon_mask], 0
    call print_item_hdr
sandwich_addon_loop:
    print_str msg_sandwich_add
    call read_line
    call atoi
    cmp eax, 0
    je sandwich_done
    cmp eax, 1
    jl sandwich_inv
    cmp eax, 2
    jg sandwich_inv
    mov ecx, eax
    dec ecx
    movzx ebx, byte [addon_mask]
    mov edx, 1
    shl edx, cl
    test ebx, edx
    jnz sandwich_duplicate
    or byte [addon_mask], dl
    mov eax, [sandwich_addons + ecx*4]
    add [total_sen], eax
    jmp sandwich_addon_loop
sandwich_duplicate:
    print_str msg_duplicate_addon
    jmp sandwich_addon_loop
sandwich_done:
    mov eax, [item_counter]
    dec eax
    mov [item_counter], eax
    cmp eax, 0
    je main_loop
    mov ecx, [saved_choice]
    dec ecx
```

```
    mov  eax, [food_prices + ecx*4]
    add  [total_sen], eax
    print_str msg_added
    jmp  sandwich_addon_start
sandwich_inv:
    print_str msg_invalid
    jmp  sandwich_addon_loop

; --- Cake ---
cake_addon_start:
    mov  byte [addon_mask], 0
    call print_item_hdr
cake_addon_loop:
    print_str msg_cake_add
    call read_line
    call atoi
    cmp  eax, 0
    je   cake_done
    cmp  eax, 1
    jl   cake_inv
    cmp  eax, 2
    jg   cake_inv
    mov  ecx, eax
    dec  ecx
    movzx ebx, byte [addon_mask]
    mov  edx, 1
    shl  edx, cl
    test ebx, edx
    jnz  cake_duplicate
    or   byte [addon_mask], dl
    mov  eax, [cake_addons + ecx*4]
    add  [total_sen], eax
    jmp  cake_addon_loop
cake_duplicate:
    print_str msg_duplicate_addon
    jmp  cake_addon_loop
cake_done:
    mov  eax, [item_counter]
    dec  eax
    mov  [item_counter], eax
    cmp  eax, 0
    je   main_loop
    mov  ecx, [saved_choice]
    dec  ecx
    mov  eax, [food_prices + ecx*4]
    add  [total_sen], eax
```

```
    print_str msg_added
    jmp  cake_addon_start
cake_inv:
    print_str msg_invalid
    jmp  cake_addon_loop

; =====
; JUICE SECTION
; =====
juice_section:
    print_str msg_juice_menu
    call read_line
    call atoi
    cmp  eax, 0
    je   main_loop
    cmp  eax, 1
    jl   juice_inv
    cmp  eax, 2
    jg   juice_inv
    mov  [saved_choice], eax
    print_str msg_qty
    call read_line
    call atoi
    cmp  eax, 0
    jle  juice_inv
    mov  [saved_qty], eax
    mov  [item_counter], eax
    mov  ecx, [saved_choice]
    dec  ecx
    mov  eax, [juice_prices + ecx*4]
    add  [total_sen], eax
    print_str msg_added
    mov  eax, [saved_choice]
    cmp  eax, 1
    je   orange_addon_start
    jmp  mango_addon_start
juice_inv:
    print_str msg_invalid
    jmp  juice_section

; --- Orange Juice ---
orange_addon_start:
    mov  byte [addon_mask], 0
    call print_item_hdr
orange_addon_loop:
    print_str msg_orange_add
```

```
    call read_line
    call atoi
    cmp  eax, 0
    je   orange_done
    cmp  eax, 1
    jl   orange_inv
    cmp  eax, 2
    jg   orange_inv
    mov  ecx, eax
    dec  ecx
    movzx ebx, byte [addon_mask]
    mov  edx, 1
    shl  edx, cl
    test ebx, edx
    jnz  orange_duplicate
    or   byte [addon_mask], dl
    mov  eax, [orange_addons + ecx*4]
    add  [total_sen], eax
    jmp  orange_addon_loop
orange_duplicate:
    print_str msg_duplicate_addon
    jmp  orange_addon_loop
orange_done:
    mov  eax, [item_counter]
    dec  eax
    mov  [item_counter], eax
    cmp  eax, 0
    je   main_loop
    mov  ecx, [saved_choice]
    dec  ecx
    mov  eax, [juice_prices + ecx*4]
    add  [total_sen], eax
    print_str msg_added
    jmp  orange_addon_start
orange_inv:
    print_str msg_invalid
    jmp  orange_addon_loop

; --- Mango Juice ---
mango_addon_start:
    mov  byte [addon_mask], 0
    call print_item_hdr
mango_addon_loop:
    print_str msg_mango_add
    call read_line
    call atoi
```

```
    cmp    eax, 0
    je     mango_done
    cmp    eax, 1
    jl     mango_inv
    cmp    eax, 3
    jg     mango_inv
    mov     ecx, eax
    dec     ecx
    movzx   ebx, byte [addon_mask]
    mov     edx, 1
    shl     edx, cl
    test    ebx, edx
    jnz     mango_duplicate
    or      byte [addon_mask], dl
    mov     eax, [mango_addons + ecx*4]
    add     [total_sen], eax
    jmp     mango_addon_loop
mango_duplicate:
    print_str msg_duplicate_addon
    jmp     mango_addon_loop
mango_done:
    mov     eax, [item_counter]
    dec     eax
    mov     [item_counter], eax
    cmp     eax, 0
    je     main_loop
    mov     ecx, [saved_choice]
    dec     ecx
    mov     eax, [juice_prices + ecx*4]
    add     [total_sen], eax
    print_str msg_added
    jmp     mango_addon_start
mango_inv:
    print_str msg_invalid
    jmp     mango_addon_loop

; =====
; WATER SECTION
; =====
water_section:
    print_str msg_water_menu
    call    read_line
    call    atoi
    cmp     eax, 0
    je     main_loop
    cmp     eax, 1
```

```
    jl  water_inv
    cmp  eax, 2
    jg  water_inv
    mov  [saved_choice], eax
    print_str msg_qty
    call read_line
    call atoi
    cmp  eax, 0
    jle  water_inv
    mov  [saved_qty], eax
    mov  [item_counter], eax
    mov  ecx, [saved_choice]
    dec  ecx
    mov  eax, [water_prices + ecx*4]
    add  [total_sen], eax
    print_str msg_added
    mov  eax, [saved_choice]
    cmp  eax, 1
    je   mineral_addon_start
    jmp  sparkling_addon_start
water_inv:
    print_str msg_invalid
    jmp  water_section

; --- Mineral Water ---
mineral_addon_start:
    mov  byte [addon_mask], 0
    call print_item_hdr
mineral_addon_loop:
    print_str msg_mineral_add
    call read_line
    call atoi
    cmp  eax, 0
    je   mineral_done
    cmp  eax, 1
    jl   mineral_inv
    cmp  eax, 2
    jg   mineral_inv
    mov  ecx, eax
    dec  ecx
    movzx ebx, byte [addon_mask]
    mov  edx, 1
    shl  edx, cl
    test ebx, edx
    jnz  mineral_duplicate
    or   byte [addon_mask], dl
```



```
    mov  eax, [mineral_addons + ecx*4]
    add  [total_sen], eax
    jmp  mineral_addon_loop
mineral_duplicate:
    print_str msg_duplicate_addon
    jmp  mineral_addon_loop
mineral_done:
    mov  eax, [item_counter]
    dec  eax
    mov  [item_counter], eax
    cmp  eax, 0
    je   main_loop
    mov  ecx, [saved_choice]
    dec  ecx
    mov  eax, [water_prices + ecx*4]
    add  [total_sen], eax
    print_str msg_added
    jmp  mineral_addon_start
mineral_inv:
    print_str msg_invalid
    jmp  mineral_addon_loop

; --- Sparkling Water ---
sparkling_addon_start:
    mov  byte [addon_mask], 0
    call print_item_hdr
sparkling_addon_loop:
    print_str msg_sparkling_add
    call read_line
    call atoi
    cmp  eax, 0
    je   sparkling_done
    cmp  eax, 1
    jl   sparkling_inv
    cmp  eax, 2
    jg   sparkling_inv
    mov  ecx, eax
    dec  ecx
    movzx ebx, byte [addon_mask]
    mov  edx, 1
    shl  edx, cl
    test ebx, edx
    jnz  sparkling_duplicate
    or   byte [addon_mask], dl
    mov  eax, [sparkling_addons + ecx*4]
    add  [total_sen], eax
```

```
        jmp  sparkling_addon_loop
sparkling_duplicate:
    print_str msg_duplicate_addon
    jmp  sparkling_addon_loop
sparkling_done:
    mov  eax, [item_counter]
    dec  eax
    mov  [item_counter], eax
    cmp  eax, 0
    je   main_loop
    mov  ecx, [saved_choice]
    dec  ecx
    mov  eax, [water_prices + ecx*4]
    add  [total_sen], eax
    print_str msg_added
    jmp  sparkling_addon_start
sparkling_inv:
    print_str msg_invalid
    jmp  sparkling_addon_loop

; =====
; EMPTY ORDER CHECK -> COUPON CHECK
; =====
show_receipt:
    mov  eax, [total_sen]
    cmp  eax, 0
    jne  coupon_check
    print_str msg_empty_order
    jmp  main_loop

; =====
; COUPON CHECK
; =====
coupon_check:
    print_str msg_coupon_q
    call read_line
    movzx eax, byte [input_buf]
    cmp  al, 'y'
    je   ask_coupon_code
    cmp  al, 'Y'
    je   ask_coupon_code
    mov  dword [coupon_type], 0
    jmp  do_receipt

ask_coupon_code:
    print_str msg_coupon_ent
```

```
    call read_line
    mov esi, input_buf
    mov edi, coupon_buf
    mov ecx, 16
    rep movsb
    ; try SAVE10
    mov esi, code_save10
    mov edi, coupon_buf
    mov ecx, 6
    repe cmpsb
    jne try_flat5
    cmp byte [coupon_buf + 6], 0
    jne try_flat5
    mov dword [coupon_type], 1
    jmp do_receipt
try_flat5:
    mov esi, code_flat5
    mov edi, coupon_buf
    mov ecx, 5
    repe cmpsb
    jne try_vip20
    cmp byte [coupon_buf + 5], 0
    jne try_vip20
    mov dword [coupon_type], 2
    jmp do_receipt
try_vip20:
    mov esi, code_vip20
    mov edi, coupon_buf
    mov ecx, 5
    repe cmpsb
    jne coupon_invalid
    cmp byte [coupon_buf + 5], 0
    jne coupon_invalid
    mov dword [coupon_type], 3
    jmp do_receipt
coupon_invalid:
    print_str msg_coupon_inv
    mov dword [coupon_type], 0

; =====
; RECEIPT
; =====
do_receipt:
    print_str msg_receipt
    print_str msg_subtotal
    mov eax, [total_sen]
```

```
    call print_rm
    ; auto-discount: 10% if subtotal >= RM100
    mov  eax, [total_sen]
    cmp  eax, 10000
    jnl  no_auto_discount
    mov  edx, 0
    mov  ecx, 10
    div  ecx
    mov  [discount_sen], eax
    print_str msg_disc_note
    print_str msg_discount
    mov  eax, [discount_sen]
    call print_rm
    jmp  check_coupon
no_auto_discount:
    mov  dword [discount_sen], 0
check_coupon:
    mov  eax, [coupon_type]
    cmp  eax, 1
    je   apply_save10
    cmp  eax, 2
    je   apply_flat5
    cmp  eax, 3
    je   apply_vip20
    mov  dword [coupon_sen], 0
    jmp  print_total
apply_save10:
    print_str msg_save10_note
    mov  eax, [total_sen]
    mov  edx, 0
    mov  ecx, 10
    div  ecx
    mov  [coupon_sen], eax
    print_str msg_coupon_hdr
    mov  eax, [coupon_sen]
    call print_rm
    jmp  print_total
apply_flat5:
    print_str msg_flat5_note
    mov  dword [coupon_sen], 500
    print_str msg_coupon_hdr
    mov  eax, [coupon_sen]
    call print_rm
    jmp  print_total
apply_vip20:
    print_str msg_vip20_note
```

```
    mov  eax, [total_sen]
    mov  edx, 0
    mov  ecx, 5
    div  ecx
    mov  [coupon_sen], eax
    print_str msg_coupon_hdr
    mov  eax, [coupon_sen]
    call print_rm
print_total:
    print_str msg_total
    mov  eax, [total_sen]
    sub  eax, [discount_sen]
    sub  eax, [coupon_sen]
    mov  [payable_sen], eax
    mov  eax, [payable_sen]
    call print_rm
    print_str msg_thankyou
    print_str msg_new_order
    print_str msg_continue

    call read_line
    movzx eax, byte [input_buf]
    cmp  al, 'n'
    je  exit_program
    cmp  al, 'N'
    je  exit_program

; Reset all values for new order
    mov  dword [total_sen], 0
    mov  dword [coupon_type], 0
    mov  dword [discount_sen], 0
    mov  dword [coupon_sen], 0

    jmp  main_loop

; =====
; _strlen: IN ESI=string  OUT EDX=length
; =====
_strlen:
    xor  edx, edx
.lp:
    cmp  byte [esi+edx], 0
    je   .done
    inc  edx
    jmp  .lp
.done:
```

```
    ret

; =====
; read_line: reads one char at a time into input_buf
; =====
read_line:
    xor    ecx, ecx
.loop:
    mov    eax, 3
    mov    ebx, 0
    mov    edx, 1
    lea    edi, [char_buf]
    push   ecx
    mov    ecx, edi
    int    0x80
    pop    ecx
    cmp    eax, 0
    jle    .done
    movzx  eax, byte [char_buf]
    cmp    al, 10
    je     .done
    cmp    al, 13
    je     .loop
    cmp    al, 0
    je     .done
    mov    [input_buf + ecx], al
    inc    ecx
    cmp    ecx, 30
    jl     .loop
.done:
    mov    byte [input_buf + ecx], 0
    ret

; =====
; atoi: input_buf -> EAX integer
; =====
atoi:
    xor    eax, eax
    xor    ecx, ecx
.lp:
    movzx  edx, byte [input_buf + ecx]
    cmp    dl, 0
    je     .done
    cmp    dl, '0'
    jl     .done
    cmp    dl, '9'
```

```
    jg     .done
    sub    dl, '0'
    imul   eax, eax, 10
    add    eax, edx
    inc    ecx
    jmp    .lp
.done:
    ret

; =====
; print_rm: IN EAX=sen Prints "X.XX\n"
; =====
print_rm:
    push   ebx
    mov    ecx, 100
    xor    edx, edx
    div    ecx
    push   edx
    call   itoa_print
    mov    eax, 4
    mov    ebx, 1
    mov    ecx, msg_dot
    mov    edx, 1
    int    0x80
    pop    eax
    call   print_2digit
    mov    eax, 4
    mov    ebx, 1
    mov    ecx, msg_newline
    mov    edx, 1
    int    0x80
    pop    ebx
    ret

; =====
; itoa_print: print EAX as unsigned decimal
; =====
itoa_print:
    lea    esi, [num_buf + 12]
    mov    byte [esi], 0
    mov    ecx, 10
.lp:
    xor    edx, edx
    div    ecx
    add    dl, '0'
    dec    esi
```

```
    mov [esi], dl
    test eax, eax
    jnz .lp
    lea edx, [num_buf + 12]
    sub edx, esi
    mov eax, 4
    mov ebx, 1
    mov ecx, esi
    int 0x80
    ret
```

```
; =====
; print_2digit: print EAX as 2-digit zero-padded decimal
; =====
print_2digit:
    mov ecx, 10
    xor edx, edx
    div ecx
    add al, '0'
    add dl, '0'
    mov [num_buf], al
    mov [num_buf+1], dl
    mov eax, 4
    mov ebx, 1
    mov ecx, num_buf
    mov edx, 2
    int 0x80
    ret
exit_program:
    mov eax, 1
    xor ebx, ebx
    int 0x80
```




COMPUTER SYSTEMS AND LOW-LEVEL TECHNIQUES

CT073-3-2 CSLLT - Individual Assignment

APD2F2509CS(AI)

YUSUF ARSLAN

Assembly Language Point of Sale System

TP081181

LECTURER: DR. HASSAN JAMIL SYED

HAND OUT DATE: 19 MARCH 2026

HAND IN DATE: 15 MAY 2026

\

Table of Contents

Table of Contents	2
1. Introduction to Assembly Language	4
1.1 What is Assembly Language?	4
1.2 x86 Architecture Overview and NASM Assembly code structure	5
2. Research and Analysis	6
2.1 Assembly Language in Industry	6
2.1.1 Embedded Systems and Firmware	6
2.1.2 Operating System Development	7
2.1.3 Performance-Critical Applications	8
2.1.4 Real-Time Systems	9
3. System Design	10
3.1 System Functionalities	10
3.2 System Flow Diagram (Flowchart)	11
3.3 System Assumptions	12
4. User Manual	13
4.1 System Requirements —	13
4.2 How to Compile and Run	13
4.3 System Walkthrough —	13
4.3.1 Welcome Screen and menu display	14
4.3.2 Item Selection , Quantity input and continue prompt	14
4.3.3 Checkout and total Display	14
4.3.4 Payment Input and Change Return	15
4.4 Error Handling Guide	15
4.4.1 item-selection value error handling	15
4.4.2 quantity-value error handling	16
4.4.3 continue-prompt input error	16
4.4.4 Insufficient payment error handling	16
5. Source Code with Explanation	17
5.2 Data Section	17
5.3 BSS Section	18
5.4 Procedures	19

5.4.1 countBytes.....	19
5.4.2 printProcedure.....	19
5.4.3 printValue	20
5.4.4 printMenu.....	20
5.4.5 readInput	21
5.4.6 convertInput	21
5.4.7 findPrice.....	22
5.4.8 calculateTotal	23
5.4.9 calculateDiscount.....	23
5.4.10 calculate_change	24
5.4.11 printInt.....	24
5.4.12 clear_register.....	25
5.5 Main Program Flow (_start).....	26
5.6 Complete Source Code	28
6. Conclusion	36
7. References.....	37

1. Introduction to Assembly Language

Modern computing systems are built on many layers of technology (programming languages). What most people see and interact with are the high-level languages such as Python and java, but underneath these easy-to-use languages exists multiple levels of transformation where this high-level code is translated into a format that the CPU could execute – 0s and 1s. Assembly sits at the lowest layer of human-readable code. The assembly programming language is an understandable representation of machine code. Unlike almost all other languages, Assembly communicates directly with the processor, making it one of the most powerful tools but also one of the most complex. Learning and understanding assembly is not an exhibition of pure knowledge, but it provides the necessary foundations for developers to understand computer architecture, large infrastructure systems programming, and hardware optimisation.

1.1 What is Assembly Language?

Assembly is a low-level programming language that has a syntactic structure where each statement represents a single machine instruction that the processor executes. Assembly allows programmers to directly communicate with the processor without using complex, raw binary or hexadecimal opcodes. Examples of these commands: MOV, ADD, SUB, JMP, LOOP – the assembler translates these commands into binary machine code, which the CPU executes. The idea is that, unlike other high-level languages, each assembly instruction directly maps to one machine instruction, which makes it very hardware efficient.

Assembly emerged in the early 1950s as an alternative to raw machine code. It was created before languages like FORTRAN and COBOL. At the time, hardware constraints made it very difficult for higher-level language compilers, which made assembly the most common tool for system and application development (Irvine, 2015). Despite enormous amounts of time passing, assembly remains relevant, whether because of its direct hardware access features, performance maximization, or minimal memory usage is needed. Assembly is inherently architecture-specific, meaning that code written for x86 cannot run on ARM without being rewritten, which is perhaps the reason behind its strong feature: speed, and core limitation: architecture inflexibility.

1.2 x86 Architecture Overview and NASM Assembly code structure

The x86 architecture is based on Intel's 8086 processor, which was released in 1978, and this architecture provided the hardware foundation that was utilized to build the system in this assignment. More specifically, this project operates in 32-bit protected mode — known as the IA-32 environment — which is what decides on the registers, memory layout, and system call mechanism that the code relies on throughout.

Registers are perhaps the most used resource in any x86 assembly program. In 32-bit mode, there are eight general-purpose registers, and each one has a role that it is conventionally expected to play.

The EAX register is the accumulator; it stores arithmetic(math) results, and when interacting with the system, it holds the system call number, which is followed by INT 0x80 to call the kernel. The EBX registers carry the first argument that is passed to the system call, and usually it is the file descriptor or whether it is the std input or std output. The Ecx register is responsible for the second argument which can be anything from a memory address to a loop counter depending on the program requirement at that moment. EDX register takes on the third argument , which is typically like a data length or the buffer size. The ESI and EDI registers are used when memory operations are involved where they usually used to store source and destination pointers. SP is what tells the system where the top of the stack currently sits, and it updates itself every time a PUSH or POP instruction runs. EBP exists to lock down the base of the current stack frame (Irvine, 2015). The key takeaway here is that there is not a single arithmetic operation, loop, or conditional branch in this system that does not run through these registers.

NASM Assembly Code is divided into 3 sections: section .data, section .bss, section .text. The .data section stores everything (variables) that have been initialized before the program runs (assigned a value) . For example, string labels, menu prompts, and constant values that the system needs during execution. The .bss section is used to store variables that will be used but have not been assigned any values, so in essence, you are just reserving memory space – it reserves blocks of memory that have no value currently stored in it and these memory blocks only store values

when the program is running. The .text section is where the actual code lives, containing every instruction the processor will execute, and it needs a global `_start` declaration at the top so the system knows exactly from where it should start executing the code (asm.sourceforge.net, n.d.).

Sys calls are the method that the program uses to ask the Linux kernel to do something for it. The way that a system call works is very straightforward: the program loads the right call number into the EAX register and places the necessary arguments into EBX, ECX, and EDX, and then what fires the call is the `INT 0x80` command, which hands control over to the kernel, so the code is executed.

2. Research and Analysis

2.1 Assembly Language in Industry

Despite the widespread usage of high-level programming languages such as Python, Java, and C++ in the modern software development industry, assembly did not go extinct; rather, according to the TIOBE index, assembly is still ranked as the 18th most globally used language as of April 2026. This reflects an existing demand in embedded systems, performance engineering, and low-level tools (TIOBE, 2026). Despite only ~ 5% of professional developers reporting using assembly regularly in Stack Overflow’s developer survey in 2025, these developers are involved in systems that billions rely on on a daily basis (Stack Overflow, 2024). Assembly continues to be used not because of its ease, but because some problems can not be solved without low-level tools. These include scenarios where a high-level code compiler does not guarantee timing precision, memory utilization, or direct hardware control. Below are some scenarios where assembly language is actively used in the industry.

2.1.1 Embedded Systems and Firmware

In modern embedded development environments, C is the most commonly used language for writing microcontroller firmware. Firmware is low-level software stored in non-volatile memory that controls how hardware starts, operates, and communicates with other components (NIST, n.d.). Despite the overwhelming prevalence of C Assembly is still present in one layer of the firmware software development process, and that is the start-up sequence. Before any C code could

run on a microcontroller (A microcontroller is a compact integrated circuit that contains a processor, memory, and input/output peripherals, allowing it to control specific functions within an embedded system (NIST, n.d.). The hardware must first be identified, basically informing the other components of the system that it exists and is running. This involves setting up the stack pointer, vector table started, and memory regions prepared. This process is called the reset handler, and it must be written in assembly because at that time the C runtime does not yet exist - there is no stack, no memory model, and no standard library to rely on (ScienceDirect, n.d.). The reset handler acts as the bridge between the microcontroller (receiving pure electricity) and the normal program, which prepares the CPU and memory, then gives control to the main application code.

Assembly is also used in Interrupt service routines (ISRs) where cycle-exact timing is needed. As noted in an ACM published research, interrupt handlers are most commonly written in assembly because, in most cases, a high-level compiler can not guarantee the precision of execution timing that the hardware needs (ACM, n.d). For example, in battery-powered IoT devices like a sensor, the device needs to wake, take an input sample, and sleep within microseconds, which is an amount of time that is too small to leave to a compiler to decide. Despite its fine-grained control outside of these two scenarios, assembly is mostly overshadowed by C in the usual embedded systems or firmware development.

2.1.2 Operating System Development

The majority of the code base of almost all modern operating systems are written almost entirely in C. The parts of an operating system written in assembly language are usually limited, but they are highly important. Assembly is mainly used in two key areas: context switching and OS porting, in these parts direct control over processor registers, stack pointers and direct hardware operations is required.

Context switching is the mechanism by which the OS saves the state of a process's register and recalls another's state – this part must be written in assembly because it requires direct manipulation of the CPU register file, which is a level of control that C does not have access too. This is supported by a Harvard University study published in the ACM journal, which confirmed that when porting operating systems across different processor architectures, thousands of lines of

assembly are written, making up the machine-dependent portion of the porting code and these lines can not be written using C, because the C programming language does not give direct control over register access and low-level stack manipulation. (Hu et al., 2021). This could be observed in the Linux kernel source code, where the file `entry_64.s` is written completely in x86 assembly. This file handles the system call entry point and the context switching logic that contributes to upholding all Linux systems running today. (Torvalds, 2024).

The above scenarios represent some of the most common uses of assembly code in operating systems.

2.1.3 Performance-Critical Applications

A performance-critical application is a system where optimization of performance is not mandatory for its functions, but makes it more complete. Assembly is used in such systems to optimize performance, and the seemingly small differences that assembly achieves when scaled up to the size of mega companies that host billions of online interactions highlight the compounded effects of these low-level optimizations.

Cryptography is one of the most straightforward examples. Many cryptographic libraries of legendary status that are well known for their performance have hand-optimized assembly code for specific performance-critical functions, especially for platform-dependent instructions like AES-NI, SSE2 or AVX, or AVX. The usage of assembly can highly improve performance (Zinzindohoue et al., 2019). The most widely deployed cryptographic library in the world, OpenSSL uses x86 assembly code to exploit processor instruction sets, which determines the optimal code path at run time based on the registered CPU's capabilities (OpenSSL, 2021). SIMD instructions are still written in C through auto-vectorisation, but hand-written assembly is still used when the absolute highest level of instruction control is needed.

Aside from cryptography, game engines and video codecs apply the same principle as needed — specific inner loops handling pixel rendering or physics are manually tuned using assembly while the rest of the codebase remains in C++. Assembly here is a very specialized optimization tool

applied to solve or improve bottlenecks, not the main foundation for the entire system. A notable exception, however, is Roller-coaster Tycoon, which was written almost entirely in x86 assembly

2.1.4 Real-Time Systems

In safety-critical real-time systems, such as automotive braking functions and systems that control medical devices, and also air traffic control software, the time at which the system executes is tightly coupled with the system's success or failure. In such systems if a correct result is delivered at the wrong time the system is considered a failure AbsInt Angewandte Informatik GmbH, 2007). Assembly's role in such systems is granular and limited to small low-level sections where direct hardware control or predictable execution time is needed. Examples include interrupt handlers, context switching and exit routines. Research on safety-critical automotive co-processor architecture confirm that brake control functions need interrupt latency to be minimized and for execution time to be have fixed tight ranges, such conditions makes it necessary to use low-level control tools for context switching and interrupt handling (Rogenmoser et al., 2024).

This is because in such systems, even a few extra clock cycles that come as a result of compiler overhead in an interrupt handler can lead to the system exceeding the time that it has to execute within, and therefore, hand-written assembly in these routines becomes a necessary part for these systems to succeed.

3. System Design

3.1 System Functionalities

No.	Feature	Description
1	Menu Display	The system displays a fixed menu of 10 items with each item's price listed at the start of each ordering attempt. The menu is printed using a loop that goes through a table of item addresses, which are stored in the .data section.
2	Item selection with input validation	The individual using the system enters a number between 1 and 10 to select an item; if the input is not in the range of accepted values, the system prints an error message and loops back to redisplay the menu
3	Quantity input with validation	After an item is selected, the user enters a quantity between 1 and 15; if the input is outside of this range the system prints an error message and re-asks the user for the quantity.
4	Total Calculation	The system internally updates the total after each Added Item. It multiplies the selected item's price by the entered quantity and adds the result to the total this allows multiple items to be added across multiple item cycles before the customer checks out.
5	Tiered Discounts	Before the user is displayed their final total, the system checks the order value and applies a flat 5-dollar discount for orders between 20 and 49 dollars, or a flat 8-dollar discount for orders of 50 dollars or more. In comparison, orders below 20 dollars receive no discount.
6	Payment and change calculation	The user enters a cash payment amount; if the amount is less than the total of the customer's purchase , the system prints an insufficient payment message and re-asks for payment. Once a valid amount is entered, the system calculates and displays the change.

Figure 3.2.1 System functionalities

3.2 System Flow Diagram (Flowchart)

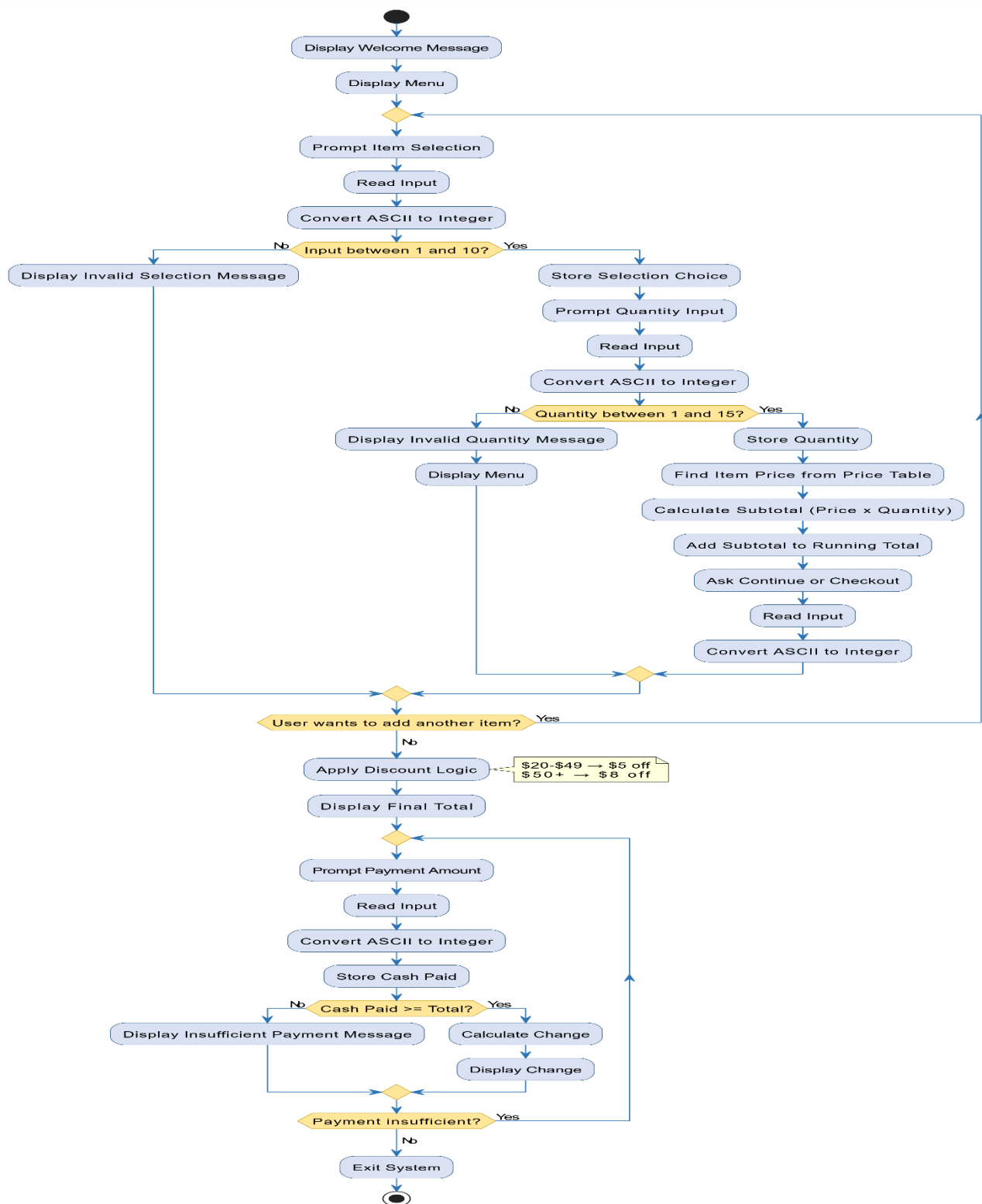


Figure 3.3.1 System Flow Diagram

3.3 System Assumptions

- The system assumes that all input is numeric. No handling for alphabetical characters was thought of. As the `convertInput` procedure processes raw ASCII bytes and only expects digit characters
- The system assumes that all monetary values are not in decimals; all transactions are done using whole numbers.
- The system assumes that it is a single-user system with only one operator interacting with the terminal. Meaning no concurrent access features.
- The system assumes a Linux operating system, with all system calls using `INT 0x80`, which is Linux-specific, and the program can not run on base Windows directly.
- The system assumes a maximum input length of 4 bytes, as input values beyond this length will overload the `user_input` variable buffer and produce noisy behaviour
- The system assumes one transaction per program execution, meaning once a checkout is complete and change is displayed, the program terminates. Meaning that a new transaction requires rerunning the program.
- The system assumes that the total price will never exceed the capacity of a 32-bit integer which is reasonable estimate for a café environment as no café sells coffee for more than 2,147,483,647 dollars ([:.](#))
- The system does not expect the operator to enter a negative value because the scenario assumes only positive integers being inputted into the system.

Figure 3.3.1 System Assumptions

4. User Manual

4.1 System Requirements —

- Linux Operating System
- NASM Assembler
- x68 Architecture 32-bit or 32-bit support libraries

4.2 How to Compile and Run

Step 1: Assemble the source file

```
nasm -f elf32 improvedMain.asm -o executable.o
```

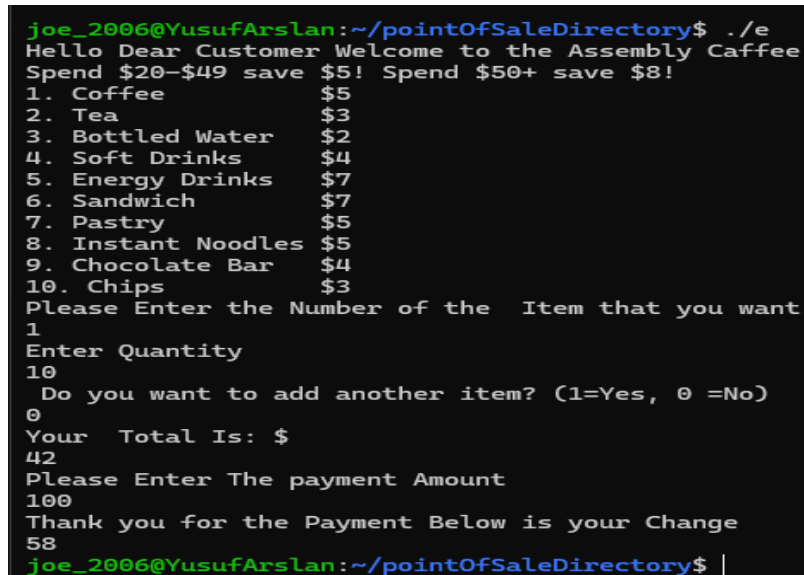
Step 2: Link the object file

```
ld -m elf_i386 executable.o -o executable
```

Step 3: Execute the program

```
./executable
```

4.3 System Walkthrough —



```
joe_2006@YusufArslan:~/pointOfSaleDirectory$ ./e
Hello Dear Customer Welcome to the Assembly Caffee
Spend $20-$49 save $5! Spend $50+ save $8!
1. Coffee $5
2. Tea $3
3. Bottled Water $2
4. Soft Drinks $4
5. Energy Drinks $7
6. Sandwich $7
7. Pastry $5
8. Instant Noodles $5
9. Chocolate Bar $4
10. Chips $3
Please Enter the Number of the Item that you want
1
Enter Quantity
10
Do you want to add another item? (1=Yes, 0 =No)
0
Your Total Is: $
42
Please Enter The payment Amount
100
Thank you for the Payment Below is your Change
58
joe_2006@YusufArslan:~/pointOfSaleDirectory$ |
```

Figure 4.3 full system Transaction

4.3.1 Welcome Screen and menu display

```
joe_2006@YusufArslan:~/pointOfSaleDirectory$ ./e
Hello Dear Customer Welcome to the Assembly Caffee
Spend $20-$49 save $5! Spend $50+ save $8!
1. Coffee $5
2. Tea $3
3. Bottled Water $2
4. Soft Drinks $4
5. Energy Drinks $7
6. Sandwich $7
7. Pastry $5
8. Instant Noodles $5
9. Chocolate Bar $4
10. Chips $3
Please Enter the Number of the Item that you want
```

Figure 4.3.1 System Startup — Welcome Screen and Menu Display

When the system starts, it shows a welcome message showing the discount thresholds and all items that are available on the menu and prompting the user to enter their selection.

4.3.2 Item Selection , Quantity input and continue prompt

```
10. Chips $3
Please Enter the Number of the Item that you want
1
Enter Quantity
10
Do you want to add another item? (1=Yes, 0 =No)
|
```

Figure 4.3.2 Item Selection, Quantity Input and Continue Prompt

After the user selects an item the user is then prompted to enter the desired quantity for that item , basically how many units of the selected item does he want. After that the user is prompted and asked on whether the user wants to add another item or checkout with 1 meaning adding another item and 0 meaning checkout.

4.3.3 Checkout and total Display

```
10
Do you want to add another item? (1=Yes, 0 =No)
0
Your Total Is: $
42
```

Figure 4.3.3 Checkout and Total Display

When the user decides to check out the system, then returns the total price of the items that the user has selected and wants to buy, this total covers all items selected with the quantities selected, meaning in the case that the user decides to add another item as possible and shown in figure 4.3.2 then the system would also keep track of the total until the user decides to checkout. After calculating the total of the items, the system internally applies any available discounts to the total.

4.3.4 Payment Input and Change Return

```
Your Total Is: $
42
Please Enter The payment Amount
100
Thank you for the Payment Below is your Change
58
joe_2006@YusufArslan:~/pointOfSaleDirectory$
```

Figure 4.3.4 payment input and change return

The system displays the final total after the discounts are applied. The user then enters the cash payment amount, and the system calculates and displays the change owed to the customer before ending the program.

4.4 Error Handling Guide

4.4.1 item-selection value error handling

```
joe_2006@YusufArslan:~/pointOfSaleDirectory$ ./e
Hello Dear Customer Welcome to the Assembly Caffee
Spend $20-$49 save $5! Spend $50+ save $8!
1. Coffee $5
2. Tea $3
3. Bottled Water $2
4. Soft Drinks $4
5. Energy Drinks $7
6. Sandwich $7
7. Pastry $5
8. Instant Noodles $5
9. Chocolate Bar $4
10. Chips $3
Please Enter the Number of the Item that you want
15
Invalid choice. Please select an item from 1 to 10
```

Figure 4.4.1 Invalid Item Selection Error Handling

4.4.2 quantity-value error handling

```
Invalid choice. Please select an item from 1 to 10
1. Coffee      $5
2. Tea         $3
3. Bottled Water $2
4. Soft Drinks $4
5. Energy Drinks $7
6. Sandwich    $7
7. Pastry      $5
8. Instant Noodles $5
9. Chocolate Bar $4
10. Chips      $3
Please Enter the Number of the Item that you want
1
Enter Quantity
100
Invalid Quantity Entered. Please Enter a quantity ranging from 1 to 15
```

Figure 4.4.2 Invalid quantity value error handling

4.4.3 continue-prompt input error

```
1
Do you want to add another item? (1=Yes, 0 =No)
30
An Invalid input has been received please enter 1 or 0
Do you want to add another item? (1=Yes, 0 =No)
```

Figure 4.4.3 Invalid continue prompt error handling

4.4.4 Insufficient payment error handling

```
Do you want to add another item? (1=Yes, 0 =No)
0
Your Total Is: $
121
Please Enter The payment Amount
30
The Amount that you have entered is not sufficient Please try paying again
Please Enter The payment Amount
```

Figure 4.4.4 Insufficient payment error

5. Source Code with Explanation

5.1 Code Structure Overview

The system's code has 3 sections section .data for variables or allocated memory that have been initialized with values before the program starts executing. The section .bss reserves memory for variables that have not been initialized . And finally the section .text section is the code that is execute with global _start with _start guiding the kernel to the start point of the program.

5.2 Data Section

```
section .data ; this is where the variables are stored
initial_greeting db "Hello Dear Customer Welcome to the Assembly Caffee", 0Ah,
db "Spend $20-$49 save $5! Spend $50+ save $8!", 0Ah, 0
wrong_continue_value_msg db "An Invalid input has been received please enter 1 or 0 ", 0Ah, 0
invalid_selection_msg db "Invalid choice. Please select an item from 1 to 10 ", 0Ah, 0
invalid_quantity_msg db "Invalid Quantity Entered. Please Enter a quantity ranging from 1 to 15", 0Ah, 0
insufficient_amount_msg db "The Amount that you have entered is not sufficient Please try paying again", 0Ah, 0
continue_prompt db " Do you want to add another item? (1=Yes, 0 =No)", 0Ah, 0
item_selection db "Please Enter the Number of the Item that you want", 0Ah, 0 ; 0 is null terminator
quantity_prompt db "Enter Quantity", 0Ah, 0 ; 0Ah - new line + 0 each is stored in a byte
subtotal_prompt db "Your Total Is: $ ", 0Ah, 0 ; new line + null terminator
cash_amount db "Please Enter The payment Amount", 0Ah, 0
change_message db "Thank you for the Payment Below is your Change", 0Ah, 0
item1_name db "1. Coffee $5", 0Ah, 0
item2_name db "2. Tea $3", 0Ah, 0
item3_name db "3. Bottled Water $2", 0Ah, 0
item4_name db "4. Soft Drinks $4", 0Ah, 0
item5_name db "5. Energy Drinks $7", 0Ah, 0
item6_name db "6. Sandwich $7", 0Ah, 0
item7_name db "7. Pastry $5", 0Ah, 0
item8_name db "8. Instant Noodles $5", 0Ah, 0
item9_name db "9. Chocolate Bar $4", 0Ah, 0
item10_name db "10. Chips $3", 0Ah, 0
menu_counter dd 10 ; used to print the menu
total_price dd 0 ; total is set to 0 initially
;price1 dd 5
;price2 dd 3
;price3 dd 2
;price4 dd 4
;price5 dd 7
;price6 dd 7
;price7 dd 5
;price8 dd 5
;price9 dd 4
;price10 dd 3

menu_items ; creating a table that stores the addresses ( of first byte ) of the items we want to print
dd item1_name
dd item2_name
dd item3_name
dd item4_name
dd item5_name
dd item6_name
dd item7_name
dd item8_name
dd item9_name
dd item10_name

menu_prices:
dd 5, 3, 2, 4, 7, 7, 5, 5, 4, 3
```

Figure 5.2 section .data

Declares all variables initialized before the program executes. Strings are defined using db, followed by the text, 0Ah (newline), and 0 (null terminator) — allowing countBytes to calculate the string length so printProcedure can print it without manual length tracking.

String Messages — initial_greeting, continue_prompt, item_selection, quantity_prompt, subtotal_prompt, cash_amount, change_message, and error messages (invalid_selection_msg,

invalid_quantity_msg, insufficient_amount_msg, wrong_continue_value_msg) are all prompts and error messages printed at various stages of execution.

Menu Item Names — item1_name through item10_name store the display string for each menu item including its price.

menu_counter — dd initialized to 10, used as a loop counter in printMenu, reset to 10 after each call.

total_price — dd initialized to 0, accumulates the running total across multiple item selections.

menu_items — a lookup table of dd addresses pointing to each item name string, used by printMenu to iterate and print each item.

menu_prices — a table of dd integers storing each item's price in the same order as menu_items, used by findPrice to retrieve the correct price based on the user's selection.

5.3 BSS Section

```
section .bss

    user_input resd 1 ;
    item_quantity resd 1 ; reserving bytes to place the input of quantity into
    selection_choice resd 1 ; reserving bytes to reserve the user choice
    selected_item_price resd 1 ; this will store the price of the selected menu item
    output_buff resb 12 ; reserving 12 bytes for this variable
    cash_paid resd 1 ; reserving 1 byte
    change resd 1 ; reserving 1 byte to store the change int
```

Figure 5.3 section .bss

The section .bss segment of the program. Declares uninitialized memory spots.

no values set, just space reserved for runtime use. Unlike .data, nothing is stored here until the program is running.

user_input, item_quantity, selection_choice — store user input after readInput and convertInput process it into integers.

selected_item_price — temporarily holds the price of the selected item for the calculation $\text{total_price} += \text{selected_item_price} * \text{item_quantity}$.

output_buff — 12 bytes used by printf to reconstruct an integer for printing, filled right to left, printed left to right.

cash_paid — stores the payment amount entered by the user.

change — stores the result of $\text{cash_paid} - \text{total_price}$, the amount returned to the customer.

5.4 Procedures

5.4.1 countBytes

```
; defining a procedure that counts the bytes of what we want to print
countBytes: ; we must move the address to esi before calling countBytes
            mov ebx, esi ; saving the address of the first byte

            ; comparison logic
            .compare: ; the . indicates that it is a sub procedure
                    cmp byte [esi], 0 ; moving forward till we reach null terminator
                    je .computeLength
                    inc esi ; move to the next byte address
                    jmp .compare ; go back to compare the next byte with null terminator
            .computeLength:
            sub esi, ebx ; subtracting the address of the null terminator - address of first byte = No of bytes in between which is the length we need
            mov edx, esi ; moving the length to edx to prepare for printing
            mov esi, ebx ; restoring back the address of the first byte into esi
            ret ; so that after calling the procedure the cpu goes back to the address of the last instruction
```

Figure 5.4.1 countBytes procedure

This procedure is a procedure that is called during the printValue procedure, the purpose of this procedure is to measure the length of the string that will be printed and store in the edx register so that the printProcedure can print the string. The address of the first byte of the string is first moved into esi then that address is copied into the ebx register: then the counting loop is defined we first compare the byte that the esi is pointed towards to check if it is equal to the null terminator, if not then we increment the address stored in esi by when so that it points to the next byte and this happens because addresses in memory are linear [1001][1002] and the loop continues until reach the null terminator then we jump to compute the length by subtracting the address of the first byte of the string stored in ebx from the address of the address of the byte that esi pointed towards which is the byte that has the null terminator, which will give us the number of bytes between the null terminator and the first byte which is the length of that string, then we moved the calculated string length into edx and restore the address of the first byte into esi since that is needed for the printProcedure to run smoothly. After esi is restored ret takes the kernel to the address of the last instruction before the procedure was called.

5.4.2 printProcedure

```
printProcedure: ; address of first byte must be moved to esi first
                mov ecx, esi ; storing address of the first byte into ecx
                mov eax, 4 ; sys call write
                mov ebx, 1 ; std output screen
                int 0x80 ; calling the kernel
                ret ; go back to the last instruction
```

Figure 5.4.2 printProcedure

This procedure is called after the address of the first byte of the string that will be printed is moved into esi and the length of the string is moved into edx which is carried out by countBytes.

This procedure moves the address of the first byte into ecx and prepares the registers for sys call
write eax $\leftarrow$ 4 (syscall number for write)

ebx $\leftarrow$ 1 (stdout)

Then int 0x80 hands the control over to the kernel which prints the value then ret sends the execution right back to the caller.

The printProcedure is always called after countBytes.

5.4.3 printValue

```
printValue:
    call countBytes
    call printProcedure ; to reduce the lines
    ret
```

Figure 5.4.3 printValue procedure

This procedure calls the countBytes and the printProcedure consecutively which ensures that printProcedure is always called after countBytes procedure.

5.4.4 printMenu

```
printMenu: ; this procedure calls other procedures
    mov edi, menu_items ; moving the first address in menu_items to edi
.loop: ; loop is subprocedure of printMenu
    mov esi, [edi] ; moving the second level address ( address of what we are going to print in esi
    call printValue
    add edi, 4 ; move the refrence to the next menu item first byte address
    dec dword [menu_counter] ; reduce the counter variable by one
    jnz .loop ; if the counter is not 0 - aka printing fininshed the go back and print
    mov dword [menu_counter], 10 ; we are resetting the menu counter in case of future needs
    ret ; after the loop retrive the address of the las instruction from stack
```

Figure 5.4.4 printMenu Procedure

This procedure prints all menu items – by looping through the menu_items table.

First the edi register is loaded with the address of the first item of the menu_item which is a table of 10 addresses with each address pointing to an item string .

In each iteration of the loop [edi] dereferences the address to reach the string address and move it to esi .

Then countBytes calculates the length into edx and then printProcedure prints it

edi is incremented by 4 moving to the next menu item , that is because each address is 4 bytes 32-bit. After that the menu_counter goes down by one if the counter is not equal to zero the loop continues until all 10 items are printed then the counter is reset to 10 for future calls then ret runs.

5.4.5 readInput

```
; so in essence we want to take that input that we received and place it into the user_input variable
readInput:
    mov eax, 3
    mov ebx, 0 ; std input - key board
    mov ecx, user_input ; this is where the value will be placed
    mov edx, 4 ; the set size for input - 4 bytes
    int 0x80
    ret
```

Figure 5.4.5 readInput Procedure

This procedure reads keyboard input from the user and stores it in user_input.

eax <--- 3 (syscall number for read)

ebx <--- 0 (stdin, keyboard)

ecx <--- address of user_input (where the input will be stored)

edx <--- 4 (max bytes to read)

int 0x80 triggers the kernel to wait for user input. Whatever the user types gets stored as raw ASCII bytes into user_input. Then ret.

5.4.6 convertInput

```
convertInput: ; this converts ascii to actual integer and the value is stored in eax
    mov esi, user_input ; we are moving the address into esi - we will use it to walk through user_input
    mov eax, 0 ; eax register will be our accumulator the one that will store the final value we built
    .loop: ; loop part used to move through every byte of user_input looking for 0Ah
        movzx ecx, byte [esi]
        cmp cl, 0Ah ; checking if we have read the last byte
        je .done
        sub cl, 48
        imul eax, eax, 10 ; we shift the value in eax by 10 so that the new value is read at its correct decimal place
        add eax, ecx ; add the int value to the accumulator
        inc esi ;
        jmp .loop
    .done:
    ret ;
```

Figure 5.4.6 convertInput Procedure

This procedure converts raw ASCII bytes that are stored in the `user_input` into an actual integer and the result is stored in `eax`.

First `esi` is loaded with the address of `user_input` and `eax` is set as 0 since it is the accumulator.

In Each iteration

- Reads one byte from `[esi]` into `ecx` using `movzx` (zero-extended, no garbage bits)
- If the byte is `0Ah` (newline), stop — that's the end of input
- Subtract 48 to convert ASCII digit to actual integer (ASCII '5' = 53, $53 - 48 = 5$)
- Shift `eax` left by multiplying by 10, then add the new digit

The `eax` register is multiplied by 10 so that each integer is at its right position.

After each loop `esi` increments each iteration to the next byte, when the `0Ah` is reached the converted ascii value is in `eax` then `ret` returns to the last cpu instruction after the procedure's call.

5.4.7 findPrice

```
findPrice:
    mov     edi, menu_prices ; now the address of the first index of menu_price is there ;
    mov     eax, [selection_choice] ; moving the price index in the table to eax register
    sub     eax, 1 ; subtracting one from eax
    imul    eax, eax, 4 ; multiplying the value in eax by 4 which is the offset to find the price index
    add     edi, eax ; adding the offset to the address of the first index / we can use eax again since the offset was moved
    mov     eax, [edi] ; moving the price into eax
    mov     dword [selected_item_price], eax ; store the price in the memory location which selected_item_price containst the address for
    ret ;
; dword is used when moving data into a memory address because memory addresses do not have an inherent size known by cpu
```

Figure 5.4.7 findPrice Procedure

The `findPrice` procedure fetches the price of the selected item from `menu_price` tables and then stores that price in the `selected item_price` variable (memory slot)

The base address (first address) of `menu_prices` is loaded into `edi` , and the value in `selection_choice` is then moved into `eax`. 1 is subtracted from `eax` to convert the offset to a 0-based offset meaning that the base address points towards the first value - because `selection_choice` stores the user's input which is 1-based and the first price already sits at the base address. In the case that the user picks item 2 this is how the offset is calculated -- $2 - 1 = 1$ then the value is multiplied by 4 which gives an offset 4 which is then added to the base address of `edi` which makes it point towards the correct price slot to land on 2nd item's price.

The offset is calculated by multiply 4 by the selection choice-1 as each price has 4 bytes reserved , then the offset is added to edi which now points at the correct price , then the value is loaded from [edi] into eax and stored in selected_item_price.

5.4.8 calculateTotal

```
calculateTotal:
    mov eax, [selected_item_price]
    mov ecx, [item_quantity]
    imul eax, ecx ; multiplying price by quantity and storing the value in esi
    add dword [total_price], eax ; moves the price into the memory location of total_price
    ret ;
```

Figure 5.4.8 calculateTotal Procedure

The calculateTotal procedure calculates the total price for a selected item. The item price which is stored in selected_item_price is moved into eax and item quantity is moved into ecx. The two values are multiplied by each other using imul and the result is stored in eax. The result of the multiplication is then added to total_price so that the user can order multiple items across multiple loop iterations which means that total_price keeps accumulating the current bill of the user. ret returns to the last cpu instruction before call.

5.4.9 calculateDiscount

```
calculateDiscount:
    mov eax, [total_price] ; moving the calculated total price into eax to figure out discount
    cmp eax, 50 ; check if the 50 dollars threshold works since 50 dollars give you a flat 8 dollars discount
    jge .discount_large ; where discount will be applied
    cmp eax, 20 ; checking if the purchase meets the first discount threshold
    jge .discount_small

    jmp .done ; in the case that there is no discount

.discount_large:
    sub dword [total_price], 8 ; applying the flat 8 dollars discount
    jmp .done
.discount_small:
    sub dword [total_price], 5 ; applying the flat 5 dollars discount
    jmp .done
.done:
    ret ; bmove back to the instruction where the procedure was called
```

Figure 5.4.9 calculateDiscount Procedure

This procedure applies a flat discount to the total price based on 2 thresholds .

The [total_price] value is first copied into eax register then these 2 comparisons are made

If $\text{eax} \geq 50 \rightarrow$ jump to .discount_large $\rightarrow$ subtract 8 from total_price

If $\text{eax} \geq 20 \rightarrow$ jump to .discount_small $\rightarrow$ subtract 5 from total_price

If neither discounts apply then jump to the .done section which basically returns to the last cpu instruction before calling the procedure. The larger procedure is checked first to make sure that it is checked for . e,g If the 20 dollars discount was checked first, a \$60 order would hit the \$20 threshold and get the smaller discount which is wrong.

5.4.10 calculate_change

```
calculate_change:
    mov eax, [cash_paid] ; moving it into accumulator
    sub eax, [total_price] ; calculating the change
    mov dword [change], eax ; storing the value into change
    ret ;
```

Figure 5.4.10 calculate_change Procedure

The calculate_change procedure stores the value of [cash_paid] into the eax register and the [total_price] value is subtracted from eax and then the remaining is returned and stored in eax which is then copied into change. The procedure does not internally manage any validation meaning that all values are appropriate before these instructions are executed.

5.4.11 printInt

```
printInt: ; we fill the buffer from last to first then print from first to last
    mov esi, output_buff + 11 ; making esi point towards the last index
    mov byte [esi], 0Ah ; making the last byte a new line
    mov ebx, 10 ; setting up our divisor
    mov eax, edi ; loading the total that we want to print into eax
    .loop:
        xor edx, edx ; cleaning edx register
        div ebx ; dividing the number by 10
        add edx, 48 ; since edx contains the remainder
        dec esi ; make esi point to the next lowest index by subtracting the address by 1
        mov [esi], dl
        cmp eax, 0 ;
        jne .loop
    .done:
        mov ecx, esi ; moving the address of the last digit into ecx since that will be starting point
        ; as we insert last to first then print first to last 3 5 4 then print 543
        mov edx, output_buff + 11 ; moving the last index address to edx
        sub edx, esi ; to calculate the length last - first = bytes in between without new line
        inc edx ; to include 0Ah in printing
        mov eax, 4 ; sys call write
        mov ebx, 1 ; std output
        int 0x80 ; kernel call
        ret ; then the value we have is stored in output_buff we just print from first index
```

Figure 5.4.11 printInt Procedure

The printInt procedure converts an integer stored in edi into an ASCII representation and prints it. Esi is pointed to the last byte of output buff where the line breaker is placed 0Ah. Eax is loaded with the value that is going to be printed and ebx is set to 10 to be the divisor.

Every iteration eax is divided by 10 and the remainder in edx has 48 added to it to convert it into ASCII and stored at the current position that esi is pointing towards and esi decrements as it points towards the left side of the buffer and this repeats until eax is 0 .

The buffer is filled right to left because division by 10 results in the least significant value first and then it is printed left to right which produces the correct order.

Once the loop finishes ecx is set to the address that esi points at and the printing happens.

5.4.12 clear_register

```
clear_registers:
    xor    eax,    eax
    xor    esi,    esi
    xor    ebx,    ebx
    xor    ecx,    ecx
    ret
```

Figure 5.4.12 clear_registers Procedure

This procedure clears out the registers and places 0s into the 32 bit so that whenever a new procedure that requires clear registers is called the clear_register procedure is called before it to ensure that the data is not corrupted due to leftover bytes.

5.5 Main Program Flow (_start)

Figure 5.5.1

```

_start:
mov esi, initial_greeting
call printValue
; the purchase loop starts from here

.purchasing_loop:
; ----- printing the Menu -----
call printMenu
; ----- asking for user selection -----
mov esi, item_selection
call printValue
; ----- receiving user item selection -----
call clear_registers
call readInput ; value stored in user_input
; ----- converting ascii value into int takes ascii from user_input and stores int after conversion in eax -----
call convertInput
cmp eax, 1
jl .invalid_selection
cmp eax, 10
jg .invalid_selection
mov dword [selection_choice], eax ; moving item number into selection choice

```

The `_start` section prints the `initial_greeting` then the `.purchasing_loop` is started where the menu shows all available items, then the user is prompted to select an item, then registers are cleared after that `readInput` procedure receives the user's input and `convertInput` then converts it into an integer. If the value is below 1 or above 10, the program jumps to `.invalid_selection`. A valid item selection is stored in `selection_choice` after it is converted to an integer.

```

; ----- displaying quantity prompt and receiving wanted quantity -----
mov esi, quantity_prompt
call printValue
call readInput
call convertInput
cmp eax, 1
jl .invalid_quantity
cmp eax, 15
jg .invalid_quantity
mov dword [item_quantity], eax
; ----- saving the current total -----
call clear_registers
call findPrice ; takes item selected and stores the price for it
call calculateTotal

; ----- asking the user if they want to continue and receiving their answers -----
.continue_prompt: ; section where prompt is provided for user to continue or not
mov esi, continue_prompt ; asking the user if they want to continue or not
call printValue
call readInput ; ascii stored in user_input
call convertInput
cmp eax, 1 ; if the user wants to continue if he does not then the jump won't happen the the system will continue execution
je .purchasing_loop
cmp eax, 0
jne .invalid_continuity_option

```

Figure 5.5.2

The quantity prompt is printed, input is read and converted — values outside 1 to 15 jump to `.invalid_quantity`. A valid quantity is stored in `item_quantity`. Registers are cleared, `findPrice` retrieves the item price, and `calculateTotal` accumulates it into `total_price`.

The continue prompt then asks the user whether to add another item. Input is read and converted — if 1, execution jumps back to `.purchasing_loop`. If 0, execution proceeds to payment. Any other value jumps to `.invalid_continuity_option`.

```
call clear_registers
mov eax, 1
int 0x80 ; calling os to terminate the execution of the code
; ----- showing a message for the wrong error if the input is wrong -----

; this section is very far below so that other sections won't fall in it accidentally
.invalid_selection:
    mov esi, invalid_selection_msg ; moving the message that will be printed
    call printValue                ; printing the message
    jmp .purchasing_loop

.invalid_quantity:
    mov esi, invalid_quantity_msg
    call printValue
    jmp .purchasing_loop

.invalid_amount:
    mov esi, insufficient_amount_msg
    call printValue
    jmp .receive_payment

.invalid_continuity_option:
    mov esi, wrong_continue_value_msg
    call printValue
    jmp .continue_prompt
```

Figure 5.5.3

Once the user finishes ordering, `calculateDiscount` is applied and `total_price` is printed. The user is prompted for payment — if the amount entered is less than `total_price`, `.invalid_amount` prints an error and loops back to `.receive_payment`. Once valid payment is received, `calculate_change` computes and prints the change. The program then exits via `int 0x80`.

The error handlers at the bottom are placed deliberately far from the main flow to prevent accidental fall-through — each prints its respective error message and jumps back to the appropriate section.

5.6 Complete Source Code

section .data ; this is where the variables are stored

```

    initial_greeting db "Hello Dear Customer Welcome to the Assembly Caffee", 0Ah,
                      db "Spend $20-$49 save $5! Spend $50+ save $8!", 0Ah, 0
    wrong_continue_value_msg db "An Invalid input has been received please enter 1 or 0 ",
0Ah, 0
    invalid_selection_msg db "Invalid choice. Please select an item from 1 to 10 ", 0Ah, 0
    invalid_quantity_msg db "Invalid Quantity Entered. Please Enter a quantity ranging from 1
to 15", 0Ah, 0
    insufficient_amount_msg db "The Amount that you have entered is not sufficient Please try
paying again", 0Ah , 0
    continue_prompt db " Do you want to add another item? (1=Yes, 0 =No)", 0Ah, 0
    item_selection db "Please Enter the Number of the Item that you want" ,0Ah , 0 ; 0 is null
terminator
    quantity_prompt db "Enter Quantity", 0Ah, 0 ; 0Ah - new line + 0 each is stored in a byte
    subtotal_prompt db "Your Total Is: $" , 0Ah, 0 ; new line + null terminator
    cash_amount db "Please Enter The payment Amount", 0Ah, 0
    change_message db "Thank you for the Payment Below is your Change", 0Ah , 0
    item1_name db "1. Coffee $5", 0Ah, 0
    item2_name db "2. Tea $3", 0Ah, 0
    item3_name db "3. Bottled Water $2", 0Ah, 0
    item4_name db "4. Soft Drinks $4", 0Ah, 0
    item5_name db "5. Energy Drinks $7", 0Ah, 0
    item6_name db "6. Sandwich $7", 0Ah, 0
    item7_name db "7. Pastry $5", 0Ah, 0
    item8_name db "8. Instant Noodles $5", 0Ah, 0
    item9_name db "9. Chocolate Bar $4", 0Ah, 0
    item10_name db "10. Chips $3", 0Ah, 0
    menu_counter dd 10 ; used to print the menu
    total_price dd 0 ; total is set to 0 initially
    ;price1 dd 5
    ;price2 dd 3
    ;price3 dd 2
    ;price4 dd 4
    ;price5 dd 7
    ;price6 dd 7
    ;price7 dd 5
    ;price8 dd 5
    ;price9 dd 4
    ;price10 dd 3

```

menu_items: ; creating a table that stores the addresses (of first byte) of the items we want to print

```

    dd item1_name
    dd item2_name
    dd item3_name

```

```
dd item4_name
dd item5_name
dd item6_name
dd item7_name
dd item8_name
dd item9_name
dd item10_name
```

```
menu_prices:
    dd 5, 3, 2, 4, 7, 7, 5, 5, 4, 3
```

section .bss

```
user_input resd 1 ;
item_quantity resd 1 ; reserving bytes to place the input of quantity into
selection_choice resd 1 ; reserving bytes to reserve the user choice
selected_item_price resd 1 ; this will store the price of the selected menu item
output_buff resb 12 ; reserving 12 bytes for this variable
cash_paid resd 1 ; reserving 1 byte
change resd 1 ; reserving 1 byte to store the change int
```

section .text

```
global _start
```

; defining a procedure that counts the bytes of what we want to print

countBytes: ; we must move the address to esi before calling countBytes
 mov ebx, esi ; saving the address of the first byte

; comparison logic

.compare: ; the . indicates that it is a sub procedure

```
    cmp byte [esi], 0 ; moving forward till we reach null terminator
```

```
    je .computeLength
```

```
    inc esi ; move to the next byte address
```

```
    jmp .compare ; go back to compare the next byte with null terminator
```

.computeLength:

sub esi, ebx ; subtracting the address of the null terminator - address of first byte = No of bytes
in between which is the length we need

```
mov edx, esi ; moving the length to edx to prepare for printing
```

```
mov esi, ebx ; restoring back the address of the first byte into esi
```

```
ret ; so that after calling the procedure the cpu goes back to the address of the last instruction
```

printProcedure: ; address of first byte must be moved to esi first

```
    mov ecx, esi ; storing address of the first byte into ecx
```

```
    mov eax, 4 ; sys call write
```

```
    mov ebx, 1 ; std output screen
```

```
int 0x80 ; calling the kernel
ret ; go back to the last instruction
```

```
printMenu: ; this procedure calls other procedures
mov edi, menu_items ; moving the first address in menu_items to edi
.loop: ; loop is subprocedure of printMenu
mov esi, [edi] ; moving the second level address ( address of what we are going to print in
esi
call printValue
add edi, 4 ; move the refrence to the next menu item first byte address
dec dword [menu_counter] ; reduce the counter variable by one
jnz .loop ; if the counter is not 0 - aka printing finished the go back and print
mov dword [menu_counter], 10 ; we are resetting the menu counter in case of future needs
ret ; after the loop retrieve the address of the las instruction from stack
```

```
printValue:
call countBytes
call printProcedure ; to reduce the lines
ret
```

; so in essence we want to take that input that we received and place it into the user_input variable

```
readInput:
mov eax,3
mov ebx, 0 ; std input - key board
mov ecx, user_input ; this is where the value will be placed
mov edx, 4 ; the set size for input - 4 bytes
int 0x80
ret
```

```
convertInput: ; this converts ascii to actual integer and the value is stored in eax
mov esi, user_input ; we are moving the address into esi - we will use it to walk through
user_input
mov eax, 0 ; eax register will be our accumulator the one that will store the final value we
built
.loop: ; loop part used to move through every byte of user_input looking for 0Ah
movzx ecx, byte [esi]
cmp cl, 0Ah ; checking if we have read the last byte
je .done
sub cl, 48
imul eax, eax, 10 ; we shift the value in eax by 10 so that the new value is read at its
correcr decimal place
add eax, ecx ; add the int value to the accumulator
inc esi ;
jmp .loop
.done:
```

ret ;

findPrice:

mov edi, menu_prices ; now the address of the first index of menu_price is there ;

mov eax, [selection_choice] ; moving the price index in the table to eax register

sub eax, 1 ; subtracting one from eax

imul eax, eax, 4 ; multiplying the value in eax by 4 which is the offset to find the price index

add edi, eax ; adding the offset to the address of the first index / we can use eax again since the offset was moved

mov eax, [edi] ; moving the price into eax

mov dword [selected_item_price], eax ; store the price in the memory location which selected_item_price contains the address for

ret ;

; dword is used when moving data into a memory address because memory addresses do not have an inherent size known by cpu

calculateTotal:

mov eax, [selected_item_price]

mov ecx, [item_quantity]

imul eax, ecx ; multiplying price by quantity and storing the value in esi

add dword [total_price], eax ; moves the price into the memory location of total_price

ret ;

calculateDiscount:

mov eax, [total_price] ; moving the calculated total price into eax to figure out discount

cmp eax, 50 ; check if the 50 dollars threshold works since 50 dollars give you a flat 8 dollars discount

jge .discount_large ; where discount will be applied

cmp eax, 20 ; checking if the purchase meets the first discount threshold

jge .discount_small

jmp .done ; in the case that there is no discount

.discount_large:

sub dword [total_price], 8 ; applying the flat 8 dollars discount

jmp .done

.discount_small:

sub dword [total_price], 5 ; applying the flat 5 dollars discount

jmp .done

.done:

ret ; bmove back to the instruction where the procedure was called

calculate_change:

```
    mov eax, [cash_paid] ; moving it into accumulator
    sub eax, [total_price] ; calculating the change
    mov dword [change], eax ; storing the value into change
    ret ;
; take note the integer that needs printing - the value itself [] should be moved to edi
printInt: ; we fill the buffer from last to first then print from first to last
    mov esi, output_buff + 11 ; making esi point towards the last index
    mov byte [esi], 0Ah ; making the last byte a new line
    mov ebx, 10 ; setting up our divisor
    mov eax, edi ; loading the total that we want to print into eax
.loop:
    xor edx, edx ; cleaning edx register
    div ebx ; dividing the number by 10
    add edx, 48 ; since edx contains the remainder
    dec esi ; make esi point to the next lowest index by subtracting the address by 1
    mov [esi], dl
    cmp eax, 0 ;
    jne .loop
.done:
    mov ecx, esi ; moving the address of the last digit into ecx since that will be starting point
                ; as we insert last to first then print first to last 3 5 4 then print 543
    mov edx, output_buff + 11 ; moving the last index address to edx
    sub edx, esi ; to calculate the length last - first = bytes in between without new line
    inc edx ; to include 0Ah in printing
    mov eax, 4 ; sys call write
    mov ebx, 1 ; std output
    int 0x80 ; kernel call
    ret ; then the value we have is stored in output_buff we just print from first index
clear_registers:
    xor eax, eax
    xor esi, esi
    xor ebx, ebx
    xor ecx, ecx
    ret

_start:
mov esi, initial_greeting
call printValue
; the purchase loop starts from here

.purchasing_loop:
;----- printing the Menu -----

    call printMenu
;----- asking for user selection -----
    mov esi, item_selection
```



```
    call printValue
;----- receiving user item selection -----
    call clear_registers
    call readInput ; value stored in user_input
;----- converting ascii value into int takes ascii from user_input
and stores int after conversion in eax -----
    call convertInput
    cmp eax, 1
    jl .invalid_selection
    cmp eax, 10
    jg .invalid_selection
    mov dword [selection_choice], eax ; moving item number into selection choice

;----- displaying quantity
prompt and receiving wanted quantity -----
    mov esi, quantity_prompt
    call printValue
    call readInput
    call convertInput
    cmp eax, 1
    jl .invalid_quantity
    cmp eax, 15
    jg .invalid_quantity
    mov dword [item_quantity], eax
;----- saving the current total -----
-----
    call clear_registers
    call findPrice ; takes item selected and stores the price for it
    call calculateTotal

;----- asking the user if they want to
continue and receiving their answers
    .continue_prompt: ; section where prompt is provided for user to continue or not
    mov esi, continue_prompt ; asking the user if they want to continue or not
    call printValue ;
    call readInput ; ascii stored in user_input
    call convertInput
    cmp eax, 1 ; if the user wants to continue if he does not then the jump won't happen the the
system will continue execution
    je .purchasing_loop
    cmp eax, 0
    jne .invalid_continuity_option
;----- Showing the user their total
after they are done with the purchase -----
    mov esi, subtotal_prompt
    call printValue
```

```
    call calculateDiscount
    mov edi, [total_price]
    call printInt
;----- asking user for
payment and receiving their payment
    .receive_payment:
        call clear_registers
        mov esi, cash_amount
        call printValue
        call clear_registers ; idk like cleaning registers hahah
; -----receiving payment starts below
    call readInput
    call convertInput
    mov dword [cash_paid], eax ; storing the chane value in cash_paid this leaves eax available
    call clear_registers
    mov eax, [total_price]
    cmp eax, [cash_paid]
    jg .invalid_amount
    call calculate_change ; after payment check chane
;----- Printing the amount of change that the user has
left -----
    call clear_registers
    mov esi, change_message
    call printValue
    ; cleaning
    mov edi, [change] ; moving change value to edi
    call printInt

call clear_registers
mov eax, 1
int 0x80 ; calling os to terminate the execution of the code
; ----- showing a message for the wrong error if the input is wrong -----
-----

; this section is very far below so that other sections won't fall in it accidentally
.invalid_selection:
    mov esi, invalid_selection_msg ; moving the message that will be printed
    call printValue ; printing the message
    jmp .purchasing_loop

.invalid_quantity:
    mov esi, invalid_quantity_msg
    call printValue
    jmp .purchasing_loop
```

```
.invalid_amount:
    mov esi, insufficient_amount_msg
    call printValue
    jmp .receive_payment

.invalid_continuity_option:
    mov esi, wrong_continue_value_msg
    call printValue
    jmp .continue_prompt
```

6. Conclusion

In this assignment, a Point of Sale System was built, which simulates the POS systems that exist in the Café. The system was built using x86 32-bit assembly language and the NASM assembler on Linux WSL. The system was correctly built, and it includes the full transaction sequences starting from the menu being displayed to calculating the change and writing it to the user. The system met the assignment requirements of implementing operation, loops, and jumps.

The system included input validation for all user entry points, a fixed discount system was applied. Also, a price lookup table was written to ease the process of price calculation. The program is made of modular procedures that are reused throughout the program. This modularity was implemented on purpose to reflect the real capabilities of assembly and simulate the workflow of a POS system.

Building this system made low-level computing concepts genuinely clear. Seeing exactly how data moves between registers, how procedure calls use the stack, how ASCII input becomes a usable integer, and how the program communicates with the Linux kernel through system calls — these are things that simply cannot be understood the same way in a high-level language.

The system has known limitations, such as single-session execution, no decimal support, and a fixed input buffer size. These are documented as assumptions in Section 3 and are considered acceptable trade-offs for a project at this level.

7. References

Irvine, K. R. (2015). *Assembly language for x86 processors* (7th ed.). Pearson Education.

NASM Project. (n.d.). Netwide Assembler (NASM). <https://www.nasm.us/>

Encyclopedia MDPI. (2022). *Intel 80386*. <https://encyclopedia.pub/entry/30733>

Irvine, K. R. (2015). *Assembly language for x86 processors* (7th ed.). Pearson Education.

Intel. (1978). *The beginning of a legend: The 8086*. Intel History Timeline.

<https://timeline.intel.com/1978/the-beginning-of-a-legend:-the-8086>

Intel. (1978). *The beginning of a legend: The 8086*. Intel History Timeline.

<https://timeline.intel.com/1978/the-beginning-of-a-legend:-the-8086>

asm.sourceforge.net. (n.d.). *Introduction to NASM: Hello world*.

<https://asm.sourceforge.net/intro/hello.html>

Stack Overflow. (2024). *Stack Overflow developer survey 2024*.

<https://survey.stackoverflow.co/2024/>

TIOBE. (2026). *TIOBE index for January 2026*. <https://www.tiobe.com/tiobe-index/>

ScienceDirect. (n.d.). *Startup code*. <https://www.sciencedirect.com/topics/engineering/startup-code>

Glossner, J., Heppner, D., Ante, S., Dubey, P., & Witnesses, G. (2003). *An optimizing assembler for explicitly parallel instruction computing (EPIC) architectures*. ACM Digital Library.

<https://dl.acm.org/doi/book/10.5555/933736>

National Institute of Standards and Technology. (n.d.). *Firmware*. Computer Security Resource

Center Glossary. <https://csrc.nist.gov/glossary/term/firmware>

National Institute of Standards and Technology. (n.d.). *Microcontroller unit*. Computer Security

Resource Center Glossary. https://csrc.nist.gov/glossary/term/microcontroller_unit

Torvalds, L. (2024). *Linux kernel source tree: arch/x86/entry/entry_64.S* [Source code]. GitHub.
https://github.com/torvalds/linux/blob/master/arch/x86/entry/entry_64.S

Hu, J., Lu, E., Holland, D. A., Kawaguchi, M., Chong, S., & Seltzer, M. I. (2021). Towards porting operating systems with program synthesis. *Journal of the ACM*, 37(4), Article 111.
<https://doi.org/10.1145/1122445.1122456>

Sawyer, C. (n.d.). *Frequently asked questions: About Chris Sawyer & game development*. Chris Sawyer Software Development.
<https://www.chrissawyergames.com/faq3.htm>](<https://www.chrissawyergames.com/faq3.htm>)

Zinzindohoue, J. K., Bhargavan, K., Protzenko, J., & Beurdouche, B. (2019). The last mile: High-assurance and high-speed cryptographic implementations. arXiv.
<https://arxiv.org/pdf/1904.04606>

OpenSSL. (2021). *OPENSSL_ia32cap — OpenSSL documentation*.
https://www.openssl.org/docs/man1.1.1/man3/OPENSSL_ia32cap.html

Rogenmoser, M., Ottaviano, A., Wistoff, N., Balas, R., Benini, L., & Gürkaynak, F. K. (2024). *SentryCore: A RISC-V co-processor system for safe, real-time control applications*. arXiv.
<https://arxiv.org/abs/2406.06546>



Asia Pacific University of Technology & Innovation

CT074-3-2 Concurrent Programming

Intake: APD2F2509CS(AI)

Yusuf Arslan

TP 081181

LECTURER: DR. MAYTHEM KAMAL ABBAS AL-ADILEE

Airport Simulation

Hand In Date : 22nd May

Table of Contents

Table of Contents.....	2
1.0 Assumptions.....	3
2. Basic requirements met.....	3
2.1 There is only 1 runway for all planes to land and depart.....	3
2.2 There can only be 3 airplanes on the airport grounds, including the runway.....	5
2.3 Each step of plane lifecycle (land, coast to gate, dock, disembark, refill, embark, undock, coast, takeoff) + Concurrent Gate Operations	6
2.4 Statistics Report:	8
3. Additional requirements met.....	9
3.1 Single Refuelling Truck constraint	9
3.2 Emergency Congested Scenario	10
4. Requirements that were not met	10
6. References	11

1.0 Assumptions

Assumptions

- 1 Landing: 1s, Coasting: 1.5s, Docking/Undocking: 0.5s, Takeoff: 1s, Gate operations: 5s total (3.5s concurrent + 1.5s embarking)
- 2 Disembarking runs concurrently with cleaning and refueling as per specification
- 3 Planes wait indefinitely outside the airport if at capacity
- 4 Emergency planes jump to the front of the landing queue
- 5 Passenger count is random between 30-50 per plane
- 6 Refueling truck travel time between gates is negligible and not simulated
- 7 Planes arrive at random intervals of 0-2 seconds as per specification
- 8 Waiting time is measured from plane arrival until landing permission is granted
- 9 All 6 planes complete one full lifecycle — denied planes wait and retry
- 10 Simulation ends when all 6 planes depart, after which statistics are printed
- 11 Gate assignment is availability-based — no reservation system
- 12 Only one plane can use the Refueling Truck at a time
- 13 Only one plane can communicate with the ATC tower at a time
- 14 Passengers boarded = passengers who embarked for departure
- 15 Gates and Runway are never accessed directly by plane threads — all allocation is mediated through the ATC

Figure 1.1 Simulation Assumptions

2. Basic requirements met

2.1 There is only 1 runway for all planes to land and depart

The single runway constraint is applied through mutual exclusion, where the ATC acts as the centralized monitor for all resource allocations. Instead of synchronizing the runway object itself, all landing and takeoff decisions are determined by the ATC's synchronized methods.

The RunwayClass shown in Figure 2.1.1 encapsulates the availability state of the runway as a boolean flag; planes never directly access the flag, but they must request permission from the ATC that evaluates the flag before giving permissions. In the case that the runway is unavailable, the plane thread calls wait() releasing the ATC lock and suspending until notifyStateChange() calls notifyAll(), waking all waiting planes to recheck

```

public class Runway { // take note that there is only one runway -
    // the runway is converted into an instance so that we can have better locking mechanisms using t
    private boolean available ;
    private int id ;
    public boolean getAvailability() {return this.available ;}
    public Runway(int id , boolean available )
    {
        this.available = available ;
        this.id = id ;
    }
    public void occupy() {available = false ;}
    public void release() {available = true ;}
}

```

Figure 2.1.1 Runway Class implementation

```

public void grantLandingPermission(AeroPlane plane)
{
    synchronized(this)
    {
        while (
            landingQueue.isEmpty() ||
            plane != landingQueue.get(index: 0) || // the planes need to be the first in the queue
            countAvailableGates() == 0 || // there should be at least one available gate
            runway.getAvailability() == false || // runway should be available
            onGroundCapacity.availablePermits() == 0 // there should be a slot - a semaphore permit
        )
        {
            try

```

Figure 2.1.2 runway availability condition in grantLandingPermission()

```

67 public synchronized void notifyStateChange() {notifyAll() ;}

```

Figure 2.1.3 notification of All planes in the wait queue via notifyStateChange()

```

        Sleep(milliseconds: 1000);
    }

    atcTower.getRunway().release(); // releasing the runway after landing
    atcTower.notifyRunwayFree(); // plane temporarily gets the lock to notify other planes that runway is free
    // the line above basically wakes up the planes in the landing queue and makes them check again
}

```

Figure 2.1.4 release of runway and notification of other planes in performLanding() method

2.2 There can only be 3 airplanes on the airport grounds, including the runway.

The 3-planes on ground capacity is enforced using the synchronized ATC block which contains a semaphore that is initialized to 3, declared in the ATC class. Inside `grantLandingPermission`, the condition `onGroundCapacity.availablePermits() == 0` is checked with other conditions, when a plane finds no available permits it calls `wait()` which releases the ATC lock and suspends the thread until it is notified. After the loop exits (meaning the conditions for landing are all true) `onGroundCapacity.acquire()` is called within the same synchronized block. To acquire a ground slot. Semaphore was chosen instead of a plain integer counter to highlight that ground capacity is a limited resource with a predefined maximum. When a plane departs, `notifyDeparture()` releases the permit and calls `notifyAll()`, waking all waiting planes to recheck.

```

8
9 public class ATC {
10     private Semaphore onGroundCapacity = new Semaphore(permits: 3) ;

```

Figure 2.2.1 Declaration and initialization of semaphore in ATC

```

public void grantLandingPermission(AeroPlane plane)
{
    synchronized(this)
    {
        while (
            landingQueue.isEmpty() ||
            plane != landingQueue.get(index: 0) || // the planes need to be the first in the queue
            countAvailableGates() == 0 || // there should be at least one available gate
            runway.getAvailability() == false || // runway should be available
            onGroundCapacity.availablePermits() == 0 // there should be a slot - a semaphore permit
        )
        {
            try
            {
                wait();
            }
            catch (InterruptedException e)
            {
                System.out.println(x: "An issue occurred while waiting for landing permission.");
                Thread.currentThread().interrupt();
                return;
            }
        }

        try
        {
            onGroundCapacity.acquire();
        }
        catch (InterruptedException e)
        {

```

Figure 2.2.2: Semaphore permit availability check and semaphore permit acquisition

```

public void releaseSlot()
{
    onGroundCapacity.release(); // release a semaphore permit
}

```

Figure 2.2.5 releaseSlot() helper ATC method

```

}
public synchronized void notifyDeparture() // this method can be called only if the pla
{
    releaseSlot(); // gives up on the planes spot to be on ground
    notifyAll();
}

```

Figure 2.2.4 notifyDeparture() synchronized ATC method

```

}
public void takeOff() {
    System.out.println("Plane-" + id + ": Taking off.");
    status = Status.TAKING_OFF;
    Sleep(milliseconds: 1000);
    atcTower.getRunway().release();
    atcTower.notifyDeparture(); // this wakes up all planes that are waiting
}

```

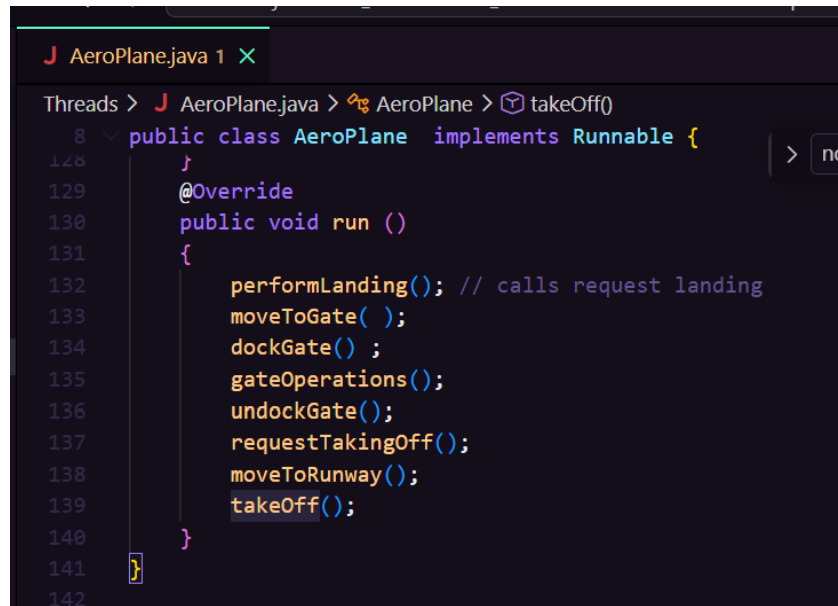
Figure 2.2.5 takeOff() AeroPlane method

2.3 Each step of plane lifecycle (land, coast to gate, dock, disembark, refill, embark, undock, coast, takeoff) + Concurrent Gate Operations

The AeroPlane class implements the Runnable interface from which it inherits the run () method . The run() method is where the Aeroplane life cycle is implemented . Each plane created is an independent concurrent thread. Multiple plane threads are executing their lifecycles at the same time which allows planes to be at different stages concurrently e.g – a plane could be taking off while another is still refueling on Gas.

Gate operations are concurrently executed. The performGateOperations() spawns three threads concurrently CleaningTask() , Disembark, and RefillSupplies() . The ATC spawns the RefuelGas Thread using requestRefuleing () and returns the thread to the Gate to manage it. All four threads run at the same time. .join() is used on Disembark to enforce a sequential execution of disembark and embark, as the Embark thread only starts after threadDisembarking.join() completes, ensuring

that passengers can not board the plane while others are still disembarking . All remaining threads are then joined before the plane proceeds to undock, guaranteeing gate operations are fully complete before the next lifecycle stage begins

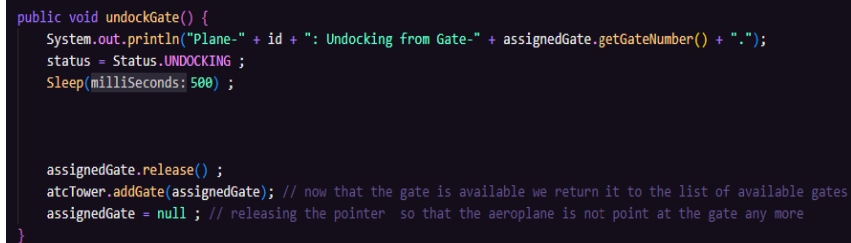


```

J AeroPlane.java 1 X
Threads > J AeroPlane.java > AeroPlane > takeOff()
8 public class AeroPlane implements Runnable {
128 }
129 @Override
130 public void run ()
131 {
132     performLanding(); // calls request landing
133     moveToGate( );
134     dockGate( );
135     gateOperations();
136     undockGate();
137     requestTakingOff();
138     moveToRunway();
139     takeOff();
140 }
141
142

```

Figure 2.3.1 run() method in AeroPlane class



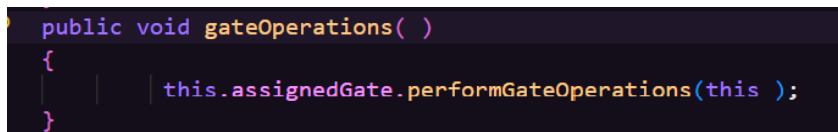
```

public void undockGate() {
    System.out.println("Plane-" + id + ": Undocking from Gate-" + assignedGate.getGateNumber() + ".");
    status = Status.UNDOCKING ;
    Sleep(milliseconds: 500) ;

    assignedGate.release() ;
    atcTower.addGate(assignedGate); // now that the gate is available we return it to the list of available gates
    assignedGate = null ; // releasing the pointer so that the aeroplane is not point at the gate any more
}

```

Figure 2.3.2 undockGate() method in AeroPlane()



```

public void gateOperations( )
{
    this.assignedGate.performGateOperations(this );
}

```

Figure 2.3.3 gateOperations() method in AeroPlane

```

public void performGateOperations(AeroPlane plane) {
    try {
        CleaningTask ct = new CleaningTask(plane);
        Disembark de = new Disembark(plane);
        RefillSupplies rs = new RefillSupplies(plane);

        Thread threadCleaning = new Thread(ct);
        Thread threadDisembarking = new Thread(de);
        Thread threadRefillSupplies = new Thread(rs);

        Thread threadRefuelGas = atcTower.requestRefueling(plane); // ATC starts it

        threadCleaning.start();
        threadDisembarking.start();
        threadRefillSupplies.start();
        // threadRefuelGas already started by ATC

        threadDisembarking.join();
        Thread threadEmbarking = new Thread(new Embark(plane));
        threadEmbarking.start();

        threadCleaning.join();
        threadRefillSupplies.join();
        threadRefuelGas.join(); // Gate waits for it
        threadEmbarking.join();
        atcTower.addPassengersBoarded(plane.getPassengerCount()); // to keep up with the statistics
    } catch (InterruptedException e) {
        System.out.println(x: "An issue occurred during gate operations.");
    }
}

```

Figure 2.3.4 performGateOperations() Gate Method

2.4 Statistics Report:

After all plane threads complete execution, the main thread calls printStatistics on the ATC object. The main thread uses join() on all plane threads before the printStatistics() is called this is used as a mechanism to block the main thread of calling the method until all of the Aeroplane threads are terminated which guarantees that all shared updates to waitingTimes and totalBoardedPassengers are fully complete before the statistics are read. Without this, the main thread could display corrupted data .

```

public void printStatistics()
{
    long minTime = waitingTimes.get(index: 0) ;
    long maxTime = waitingTimes.get(index: 0) ;
    long sum = 0; // to calculate the average waiting time
    for (Long i:waitingTimes )
    {
        if ( i < minTime )
        {
            minTime = i ;
        }
        if (i > maxTime )
        {
            maxTime = i ;
        }
        sum += i ;
    } // type conversions are weird in java
    double avgTime = (double)sum / waitingTimes.size() ;
    System.out.println("The number of empty Gates is: "+ availableGates.size() );
    System.out.println("The maximum waiting time across all planes is: "+ maxTime/1000.0 + " s" );
    System.out.println("The minimum waiting time across all planes is: "+ minTime/1000.0 + " s" );
    System.out.println("The Average waiting Time of a plane is: "+ avgTime/1000.0 + " s" );
    System.out.println("The total number of planes served is: "+ waitingTimes.size() );
    System.out.println("The total number of passengers boarded is: "+ totalBoardedPassengers );
}

```

Figure 2.4.1 printStatistics() method in ATC

```

public class Main
{
    public static void main (String [] args)
    {
        // 6 planes are gonna be generated randomly
        // before the statistics every plan must finish first
        for (Thread t: threads )
        {
            try {t.join() ; } catch (InterruptedException e ) {System.out.println(x: "An Issue occurred while waiting for all threads to finish to print the statistics "); }
        }
        airportWatchTower.printStatistics(); // after everything runs
    }
}

```

Figure 2.4.2 printStatistics call in Main after Threads Joining

```

public synchronized void addPassengersBoarded(int passengers) {this.totalBoardedPassengers += passengers ; }

```

Figure 2.4.3 addPassengersBoarded() synchronized method in ATC

3. Additional requirements met

3.1 Single Refuelling Truck constraint

In This simulation, only one refuel truck exists, and it should serve the 3 gates . This constraint was implemented using a lock on the RefuelTruck object inside the RefuelGas.run() method. The synchronized(fuelTruck) block uses the shared Refuel Truck object as a monitor – meaning that only the thread that acquires the lock on the ruck object can commence with the refueling operations. All other threads attempting that try to refuel while the truck is occupied will be blocked until the lock is released . The RefuelGas thread is created and started by the ATC through the requestRefueling() method which returns the thread back to the Gate so that the Gate coordinates the Refueling with other concurrent gate operations that are going on .

```

Threads > J RefuelGas.java > RefuelGas
4 public class RefuelGas implements Runnable {
13
14 @Override
15 public void run() {
16     synchronized(fuelTruck) {
17         System.out.println("Plane-" + plane.getId() + ": Refueling Gas.");
18         try {
19             Thread.sleep(millis: 2000);
20         } catch (InterruptedException e) {
21             System.out.println(x: "An accident occurred during plane refueling.");
22         }
23     }
24 }

```

Figure 3.1.1 synchronized block in RefuelGas run() method

```

}
public Thread requestRefueling(AeroPlane plane) {
    Thread threadRefuelGas = new Thread(new RefuelGas(this.truck, plane));
    threadRefuelGas.start();
    return threadRefuelGas; // return thread to Gate for joining
}

```

Figure 3.1.2 requestRefueling() method in ATC

3.2 Emergency Congested Scenario

The emergency landing scenario is implemented using a priority queue insertion inside the synchronized method `addPLaneLanding()`. When an emergency plane call this method it is placed at the 0th index of the `LandingQueue` instead of the end, which means that the Emergency plane jumps ahead of all waiting planes. Unlike in normal scenarios where the queue follows a FIFO order (first in first out order). The synchronized keyword ensures that this insertion is thread safe – meaning that no 2 planes can modify the `LandingQueue` at the same time. The condition `plane != landingQueue.get(0)` inside the `grantLandingPermission()` method enforces that only the first plane in the queue can proceed to ask check for permission to land meaning that in an emergency scenario the emergency plane will be the first in the `LandingQueue` to receive landing permissions regardless of how many planes are already lining up in the queue.

```
public synchronized void addPlaneLanding(AeroPlane plane)
{
    if (plane.getEmergencyStatus() )
    {
        System.out.println("ATC: Emergency landing detected for Plane-" + plane.getId() + ". Jumping to front of queue.");
        landingQueue.add(index: 0, plane); // move the plane to the front
    }
    else
    {
        landingQueue.add(plane) ;
    }
    // after a plane joins the queue it notifies other planes to check
    notifyAll();
}
```

Figure 3.2.1 `addPLaneLanding()` prioritizing emergency plane in the queue .

```
}
public void grantLandingPermission(AeroPlane plane)
{
    synchronized(this)
    {
        while (
            landingQueue.isEmpty() ||
            plane != landingQueue.get(index: 0) || // the planes need to be the first in the queue
            countavailableGates() == 0 || // there should be at least one available gate
            runway.getAvailability() == false || // runway should be available
            onGroundCapacity.availablePermits() == 0 // there should be a slot - a semaphore permit
        )
        {
            try
            {
                wait();
            }
            catch (InterruptedException e)
            {
                // ...
            }
        }
    }
}
```

Figure 3.2.2 `plane != landingQueue.get(0)` Condition in `grantLandingPermission()` -ATC

4. Requirements that were not met

ALL REQUIREMENTS WERE FULFILLED

6. References

- Baeldung, 2024. How to use wait() and notify() in Java. [online] Available at: <https://www.baeldung.com/java-wait-notify> [Accessed May 2026].
- Baeldung, 2024. Why wait() requires synchronization. [online] Available at: <https://www.baeldung.com/java-wait-necessary-synchronization> [Accessed May 2026].
- Jenkov, J., 2024. Thread signaling in Java. [online] Available at: <https://jenkov.com/tutorials/java-concurrency/thread-signaling.html> [Accessed May 2026].
- Jenkov, J., 2021. Deadlock prevention in Java. [online] Available at: <https://jenkov.com/tutorials/java-concurrency/deadlock-prevention.html> [Accessed May 2026].
- Magee, J. and Kramer, J., 2015. Concurrency: monitors and condition synchronization. Imperial College London. [online] Available at: <https://www.doc.ic.ac.uk/~jnm/book/pdf/ch5.pdf> [Accessed May 2026].
- Prut, A., 2020. Race conditions, locks, semaphores and deadlocks. Medium. [online] Available at: <https://medium.com/swlh/race-conditions-locks-semaphores-and-deadlocks-a4f783876529> [Accessed May 2026].
- Tiset, S., 2025. Understanding mutexes and semaphores: preventing deadlocks and starvation. Medium. [online] Available at: <https://medium.com/@sylvain.tiset/understanding-mutexes-and-semaphores-preventing-deadlocks-and-starvation-in-concurrent-programming-6bc477b7b7a9> [Accessed May 2026].
- Obsidian Scheduler, n.d. Java concurrency part 1 — semaphores. [online] Available at: <https://web.obsidianscheduler.com/java-concurrency-part-1-semaphores/> [Accessed May 2026].
- Oracle, n.d. Java SE documentation: Thread. [online] Available at: <https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html> [Accessed May 2026].
- Goetz, B., 2006. Java concurrency in practice. Addison-Wesley Professional.